



Tunisian Republic
Ministry of Higher Education and Scientific
Research
University of Sousse
Higher Institute of Computer Science
and Communication Technologies
(ISITCOM)



End of Studies Project Report

Submitted in partial fulfillment of the requirements for the
National Engineering Degree — Specialty: Teleinformatics

Presented by:

Asma Daoussi

Development of a Digital Asset Marketplace

for Inherited Games Studio (IGS)

Carried out at:



Inherited Games Studio

Academic Supervisor: Mr Sami BEN AMOR

Company Supervisor: Mrs. Nour LTAIEF

Academic Year: 2025–2026

Authorization for Defense

Company Supervisor

I hereby authorize the student to submit their End of Studies Project Report for the jury defense.

Mrs Nour LTAIEF

Company Supervisor, Inherited Games Studio

Signature: _____

Academic Supervisor

I hereby authorize the student to submit their End of Studies Project Report for the jury defense.

Mr Sami BEN AMOR

Academic Supervisor, ISITCOM — University of Sousse

Signature: _____

Dedication

To my beloved family,

*who have been my unwavering pillar of strength
throughout every step of this journey.*

To my parents,

*whose sacrifices, endless encouragement, and unconditional love
have been the foundation upon which I built every dream.*

*To my brother Oussema and my sister Emna,
for the laughter, the support, and the belief
that always pushed me forward.*

To my friends and colleagues,

*who made these long study years memorable
and filled them with moments of joy and solidarity.*

And to myself —

*for the late nights, the persistence, and the courage
to see this through to the end.*

This work is for all of you.

Asma Daoussi

Acknowledgments

First and foremost, I would like to express my sincere gratitude to **Mr. Sami Ben Amor**, my academic supervisor, for his continuous guidance, insightful advice, and unwavering support throughout the realization of this project. His expertise and constructive feedback have been invaluable in shaping both the technical direction and the quality of this work.

I extend my heartfelt thanks to **Mrs Nour Ltaief**, my company supervisor at Inherited Games Studio, for welcoming me into the team, sharing her professional experience, and providing the resources and environment necessary to bring this project to life. Her trust in my capabilities and her constant encouragement made this internship an enriching and rewarding experience.

I would like to express my appreciation to all the staff at **Inherited Games Studio (IGS)** for their warm welcome and their readiness to assist me whenever needed. Working alongside such a talented and passionate team has been a privilege.

My gratitude also goes to the faculty and staff of **ISITCOM — Higher Institute of Computer Science and Communication Technologies** at the **University of Sousse**, whose dedication to quality education provided me with the theoretical and practical foundations that made this project possible.

I am also deeply thankful to the members of the **jury** for dedicating their time to evaluate this work, and for the honor they grant me by participating in its defense.

Finally, I would like to thank my family and friends for their patience, encouragement, and moral support during the long nights and challenging moments of this project.

Asma Daoussi
Sousse, 2026

Contents

Dedication	i
Acknowledgments	ii
List of Figures	vii
List of Tables	viii
List of Acronyms	ix
General Introduction	1
1 General Context	3
1.1 Presentation of the Host Organization	4
1.2 Project Presentation	4
1.2.1 Context and Motivation	4
1.2.2 Problem Statement	5
1.3 Comparative Study of Existing Solutions	5
1.3.1 Analysis	5
1.3.2 Platform Comparison	5
1.3.3 Comparison table of platforms	7
1.3.4 Proposed Solution	7
1.4 Work Methodology	8
1.4.1 Comparative Study of Methodologies	8
1.4.2 Methodology Adopted: Scrum	10
1.4.3 Scrum Process Roles	10
1.4.4 Implementation of the Scrum Methodology	11
1.5 Planning	11
2 Requirements Analysis	13
2.1 Actors Identification	14
2.2 Needs Identification	14
2.2.1 Functional Requirements	15
2.3 Non-Functional Requirements	17
2.4 Project Management with Scrum	17

2.4.1	Product Backlog	17
2.5	Project Architecture: 3-Tier	19
2.6	Design Patterns	20
2.7	Work Environment	21
2.7.1	Hardware Environment	21
2.7.2	Software Environment	21
2.7.3	Frameworks and Libraries	23
2.7.4	Programming Languages	24
2.7.5	Data Persistence and Cloud Services	24
2.8	Global Use Case Diagrams	26
2.8.1	Marketplace — Use Case Diagram	26
2.8.2	Workspace — Use Case Diagram	26
2.9	Global Class Diagrams	27
2.9.1	Marketplace — Class Diagram	27
2.9.2	Workspace — Class Diagram	28
3	Marketplace Foundations	30
3.1	Sprint 1: Authentication, Profiles and RBAC	31
3.1.1	Sprint 1 Backlog	31
3.1.2	Sprint 1 Analysis	31
3.1.3	Sprint 1 Realization	35
3.1.4	Sprint 1 Conclusion	37
3.2	Sprint 2: Catalog, Cart, Stripe Payment and Library	37
3.2.1	Sprint 2 Backlog	38
3.2.2	Sprint 2 Analysis	38
3.2.3	Sprint 2 Realization	43
3.2.4	Sprint 2 Conclusion	46
4	Seller Workflow and Workspace	47
4.1	Sprint 3: Asset Upload, Moderation, Reviews and Seller Workflow	48
4.1.1	Sprint 3 Backlog	49
4.1.2	Sprint 3 Analysis	49
4.1.3	Sprint 3 Realization	53
4.1.4	Sprint 3 Conclusion	55
4.2	Sprint 4: IGS Workspace and Administration	56
4.2.1	Sprint 4 Backlog	56
4.2.2	Sprint 4 Analysis	56
4.2.3	Sprint 4 Realization	60
4.2.4	Sprint 4 Conclusion	63

5	Advanced Features	64
5.1	Sprint 5 Backlog	65
5.2	AI Services	66
5.2.1	AI Chatbot	66
5.2.2	Review Sentiment Analysis	67
5.2.3	Personalized Recommendations	69
5.3	Security Hardening	70
5.4	VR/AR Immersive Preview	71
5.5	Planned Deployment on Hostinger VPS	71
5.5.1	Target Infrastructure	71
5.5.2	Key Deployment Concepts	72
5.5.3	Planned CI/CD Pipeline	72
	General Conclusion	74
	Webography	75
	Abstract	78

List of Figures

1.1	Inherited Games Studio Logo	4
1.2	Unity Asset Store [1]	5
1.3	Fab Marketplace [2]	6
1.4	Sketchfab Platform [3]	6
1.5	CGTrader Marketplace [4]	7
1.6	Conceptual architecture of the proposed solution	8
1.7	Scrum process overview	10
1.8	Scrum Team Roles	10
1.9	ClickUp task board	11
1.10	Project Timeline — 18-week Gantt chart across five Scrum sprints . . .	11
2.1	User Role Hierarchy	14
2.2	Three-Tier Architecture	19
2.3	Use Case Diagram — IGS Marketplace	26
2.4	Use Case Diagram — IGS Workspace	27
2.5	Class Diagram — IGS Marketplace	28
2.6	Class Diagram — IGS Workspace	29
3.1	Use case diagram — Sprint 1	32
3.2	Sequence diagram — Authentication flow	34
3.3	Class diagram — Sprint 1: Authentication & RBAC	35
3.4	RBAC role and permission hierarchy	36
3.5	Login and Sign-Up interface (placeholder)	36
3.6	Profile management interface (placeholder)	37
3.7	Use case diagram — Sprint 2	39
3.8	Sequence diagram — Stripe payment to library unlock	41
3.9	Sequence diagram — Intern asset request	42
3.10	Class diagram — Sprint 2: Catalog, Cart, Payment & Library	43
3.11	Catalogue browsing interface (placeholder)	45
3.12	Asset detail page with interactive 3D preview (placeholder)	45
3.13	Cart and Stripe checkout (placeholder)	46
3.14	Personal download library (placeholder)	46
4.1	Use case diagram — Sprint 3	50

4.2	Sequence diagram — Seller upload to moderation queue	52
4.3	Class diagram — Sprint 3: Upload, Moderation & Reviews	53
4.4	Asset upload form (placeholder)	54
4.5	Moderation queue interface (placeholder)	55
4.6	Review list and rating widget (placeholder)	55
4.7	Creator Studio dashboard (placeholder)	55
4.8	Trade composer and incoming requests (placeholder)	55
4.9	Use case diagram — Sprint 4 (Workspace)	57
4.10	Sequence diagram — Intern asset request and supervisor approval . . .	59
4.11	Class diagram — Sprint 4: IGS Workspace	60
4.12	IGS Workspace dashboard (placeholder)	62
4.13	Kanban task board (placeholder)	62
4.14	Workspace moderation page (placeholder)	62
4.15	Audit-log viewer (placeholder)	62
4.16	Roles and Permissions page (placeholder)	63
5.1	AI chatbot widget — intent-aware assistant available on every page . .	67
5.2	Sentiment analysis — review badges (left) and seller aggregate (right) .	68
5.3	Recommendations page — personalized (warm user) and trending (cold start)	70
5.4	WebXR immersive preview — 3D model inspected in VR on Meta Quest 3	71

List of Tables

1.1	Comparison of existing 3D asset marketplace platforms	7
1.2	Comparison of development methodologies	9
2.1	Non-Functional Requirements	17
2.2	Product Backlog — IGS Marketplace	17
3.1	Sprint 1 Backlog	31
3.2	Use Case Description: Register and Authenticate	33
3.3	Role hierarchy and access levels	36
3.4	Sprint 2 Backlog	38
3.5	Use Case Description: Purchase an Asset	40
3.6	Asset type to preview component mapping	44
3.7	Payment methods	45
4.1	Sprint 3 Backlog	49
4.2	Use Case Description: Upload and Submit an Asset	51
4.3	Sprint 4 Backlog	56
4.4	Use Case Description: Workspace Project and Task Management . . .	58
4.5	Workspace sidebar sections and access requirements	61
5.1	Sprint 5 Backlog	65
5.2	Intent categories handled by the classifier	66
5.3	Sentiment classification — decision matrix	68
5.4	User interaction signals and weights	69

List of Acronyms

Acronym	Definition
API	Application Programming Interface
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DB	Database
DM	Direct Message
ERP	Enterprise Resource Planning
GLB	GL Binary (3D format)
GLTF	GL Transmission Format
HDRI	High Dynamic Range Imaging
HTTP	HyperText Transfer Protocol
IGS	Inherited Games Studio
ISITCOM	Institut Supérieur de l'Informatique et des Technologies de Communication
JWT	JSON Web Token
MVC	Model-View-Controller
OAuth	Open Authorization
ORM	Object-Relational Mapping
PBR	Physically Based Rendering
PFE	Projet de Fin d'Études (End of Studies Project)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
SSO	Single Sign-On
UML	Unified Modeling Language
UUID	Universally Unique Identifier

General Introduction

The global video game industry has experienced remarkable growth over the past decade, evolving from a niche entertainment segment into one of the most lucrative creative economies in the world. With revenues surpassing hundreds of billions of dollars annually and a continuously expanding ecosystem of developers, studios, and independent creators, the demand for high-quality digital assets has never been greater. Beyond the games themselves, a significant and often overlooked opportunity lies not in the creation of video games directly, but in building the infrastructure that supports them, namely, digital marketplaces dedicated to the exchange of 3D assets. These platforms serve as essential bridges between creators and consumers, enabling studios, developers, architects, and designers to access a vast library of ready-made models, textures, animations, and environments. The rise of real-time 3D technologies, game engines such as Unity and Unreal Engine, and the expanding metaverse economy has further accelerated the need for structured, accessible, and reliable 3D asset marketplace solutions. Investing in such platforms represents a strategic and high-value proposition in today's digital economy, as they reduce production costs, shorten development cycles, and democratize access to professional-grade visual content.

For this purpose, this project introduces a unified 3D digital marketplace — a centralized platform designed to connect creators and consumers of 3D assets within a single ecosystem. The platform is built as a two-sided system: a public-facing **IGS Marketplace** where external users can browse, upload, purchase, and sell assets across multiple categories, and a private **IGS Workspace** reserved for internal company staff, covering project management, team collaboration, content moderation, and administrative control. By combining both sides under one shared backend, the solution addresses the fragmentation of the 3D asset market while also providing IGS with the internal tooling it needs to operate the platform efficiently.

This report is structured as follows:

- **Chapter 1** presents the host organization, the project context, a comparative study of existing solutions, and the chosen Scrum methodology.
- **Chapter 2** defines the system actors, functional and non-functional requirements, product backlog, and logical architecture.
- **Chapter 3 — Marketplace Foundations** covers Sprints 1 and 2: authentication, RBAC, the public asset catalog, 3D and WebXR previews, cart, payment,

and download library.

- **Chapter 4 — Seller Workflow and Workspace** covers Sprints 3 and 4: asset upload, moderation, reviews, Creator Studio, and the full internal IGS Workspace.
- **Chapter 5 — Advanced Features** covers Sprint 5: the AI chatbot, sentiment analysis, recommendations, security hardening, WebXR immersive preview, and the planned deployment.

General Context

Sommaire

1.1	Presentation of the Host Organization	4
1.2	Project Presentation	4
1.2.1	Context and Motivation	4
1.2.2	Problem Statement	5
1.3	Comparative Study of Existing Solutions	5
1.3.1	Analysis	5
1.3.2	Platform Comparison	5
1.3.3	Comparison table of platforms	7
1.3.4	Proposed Solution	7
1.4	Work Methodology	8
1.4.1	Comparative Study of Methodologies	8
1.4.2	Methodology Adopted: Scrum	10
1.4.3	Scrum Process Roles	10
1.4.4	Implementation of the Scrum Methodology	11
1.5	Planning	11

Introduction

This chapter lays the foundation of the project. It starts with presenting the host organization and the project context then states the problem followed by a comparative study of existing solutions. Then it describes the proposed solution and its conceptual architecture. It also outlines the project planning. The chapter closes with the work methodology, with a focus on the comparative study of development methodologies and the reason behind the choice of Scrum

1.1 Presentation of the Host Organization

Inherited Games Studio (IGS) is a game development studio that creates educational and interactive games using immersive technologies and AI-driven mechanics.

A. Mission

IGS develops games that combine learning with entertainment, using VR, AR and AI to create engaging experiences for users.



Figure 1.1: Inherited Games Studio Logo

B. Methodology

IGS employs several key strategies: immersive VR/AR environments for spatial learning, AI systems that adapt gameplay to player behavior, multiplayer and gamification for engagement, data-driven refinement, and balanced edutainment design.

1.2 Project Presentation

1.2.1 Context and Motivation

The global digital-content economy has entered a phase of sustained growth. Recent market research [35] values the worldwide **3D model marketplace** at approximately **USD 1.92 billion in 2023**, with a forecast of **USD 4.86 billion by 2031** and a compound annual growth rate of **11.2%**. A complementary study [34] attributes

most of this demand to three industry verticals: video games (**38.2%** of consumption), architectural visualization (**22.1%**) and film and television production (**17.8%**).

1.2.2 Problem Statement

Despite this global trajectory, Tunisia did not have any dedicated 3D digital asset marketplace. Local developers and studios were forced to rely on foreign platforms.

Beyond the market gap, IGS itself lacked a centralized internal system to manage its own assets, coordinate employee activities, supervise trainees and monitor platform operations. Critical tasks were handled through disconnected tools, creating friction and reducing traceability across the organization.

The project addresses two intertwined problems. The first is external: the absence of a national platform through which Tunisian creators can publish, price and sell three-dimensional content while receiving payment in local currency. The second is internal: the absence within IGS of an integrated management environment that governs asset workflows, access rights, team collaboration and operational visibility under a single, coherent system.

1.3 Comparative Study of Existing Solutions

1.3.1 Analysis

The international landscape is dominated by several major platforms.

1.3.2 Platform Comparison

Unity Asset Store

Launched in 2010, Unity Asset Store is tightly integrated with the Unity Editor, offering millions of assets. However, it is engine-specific, lacks RBAC, and provides no internal team tools.

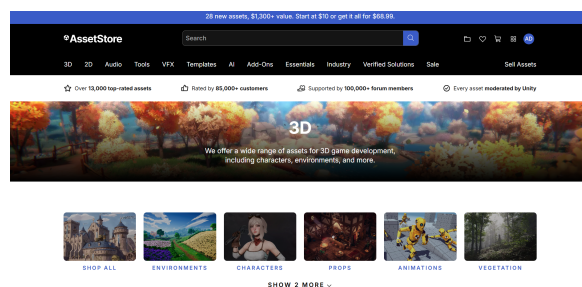


Figure 1.2: Unity Asset Store [1]

Fab (Epic Games)

Fab, Epic Games' unified marketplace, supports multiple engines and benefits from Unreal Engine integration. Despite its breadth, it lacks customization, internal workflows, and AI-powered features.

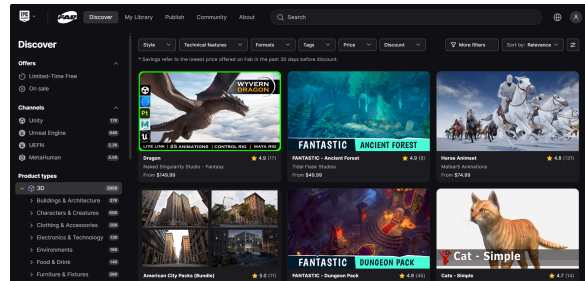


Figure 1.3: Fab Marketplace [2]

Sketchfab

Sketchfab excels in in-browser 3D visualization and supports multiple formats. It is widely used for showcasing models but does not provide monetization flexibility, RBAC, or internal studio management.

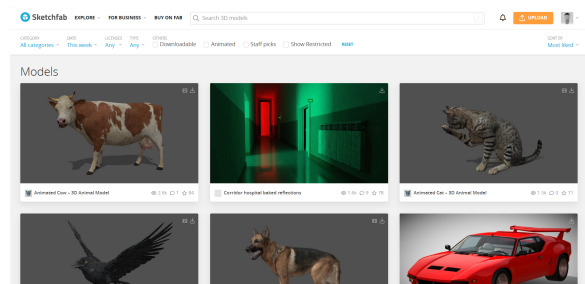


Figure 1.4: Sketchfab Platform [3]

CGTrader

CGTrader offers a large catalog and supports multiple engines. However, it is externally hosted, lacks internal collaboration tools, and does not integrate AI or role-based workflows.

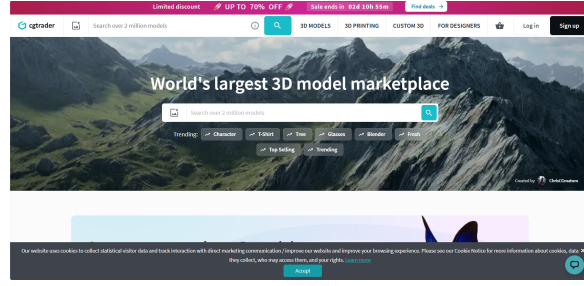


Figure 1.5: CGTrader Marketplace [4]

1.3.3 Comparison table of platforms

Table 1.1: Comparison of existing 3D asset marketplace platforms

Feature	Unity	Fab	Sketchfab	CGTrader	IGS Marketplace
Multi-engine support	×	✓	✓	✓	✓
In-browser 3D preview	Limited	Limited	✓	✓	✓
WebXR / VR-AR preview	×	×	×	×	✓
Custom RBAC	×	×	×	×	✓
Internal team tools	×	×	×	×	✓
AI recommendation	×	×	×	×	✓
AI chatbot	×	×	×	×	✓
Intern access control	×	×	×	×	✓
Local payment gateway	×	×	×	×	✓
Open/self-hosted	×	×	×	×	✓

1.3.4 Proposed Solution

The project developed for IGS consists in a unified digital platform dedicated to the exchange and management of three-dimensional creative assets. It was built as a **two-sided system**.

The first side, called **IGS Marketplace**, is a public-facing storefront open to visitors, buyers and independent creators. It allows external sellers to publish and monetize their work and gives buyers a structured catalog covering thirteen asset categories, with secure transactions and a personal download library. A distinctive feature of this storefront is its **immersive preview capability**: beyond a conventional thumbnail, visitors can manipulate a three-dimensional asset interactively inside the browser and step into the asset in **virtual reality / augmented reality**.

The second side, called **IGS Workspace**, is a private dashboard reserved for company collaborators. It supports the internal life of the studio through project management, team communication, supervised onboarding of trainees, content moderation and access to a full suite of administrative tools. It sits at the intersection of an **ERP** — governing internal processes and resources — and a **CRM** — managing relationships and communication with the platform’s user community.

Although the two interfaces serve different audiences, they share a common foundation that unifies user identity, access-control logic and the data layer. The platform additionally integrates intelligent components — a conversational assistant, an automated review analyser and a personalized recommendation engine — that enrich the user experience without altering the core business workflows.

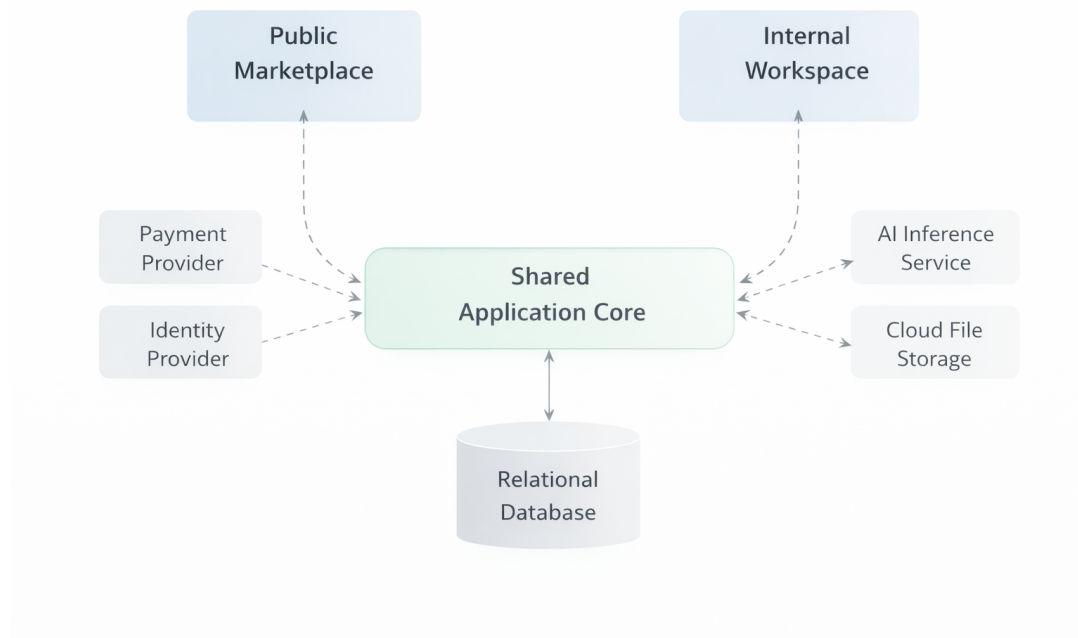


Figure 1.6: Conceptual architecture of the proposed solution

1.4 Work Methodology

1.4.1 Comparative Study of Methodologies

Before selecting a development approach, three widely adopted software engineering methodologies were examined: the Waterfall model, the V-Cycle model, and the Agile Scrum framework. Each carries distinct assumptions about requirements stability, feedback cycles, and risk management.

A. Waterfall Model

The Waterfall model is a **sequential, plan-driven** approach in which each phase (requirements, design, implementation, testing, deployment) must be completed before the next one begins. All requirements are frozen at the outset, making it poorly suited to projects where scope evolves during development.

B. V-Cycle Model

The V-Cycle extends Waterfall by pairing each development phase with a corresponding testing phase. Testing is planned early, improving defect traceability. However, it inherits the same rigidity as Waterfall and delivers a working product only at the end.

C. Agile Scrum Framework

Scrum is an **iterative and incremental** framework built on short development cycles called **sprints** (typically two to four weeks). At the end of each sprint a shippable product increment is delivered and reviewed by stakeholders. Three core roles structure the team: *Product Owner*, *Scrum Master*, and *Development Team*.

D. Side-by-Side Comparison

Table 1.2: Comparison of development methodologies

Criterion	Waterfall	V-Cycle	Agile Scrum
Requirements	Fixed upfront	Fixed upfront	Evolving backlog
Delivery	Single final release	Single final release	Incremental sprints
Stakeholder feedback	At delivery only	At delivery only	Every sprint
Testing integration	End of project	Paired with each phase	Continuous (per sprint)
Flexibility to change	Very low	Low	High
Risk management	Late detection	Earlier via test plans	Continuous mitigation
Best suited for	Stable, well-defined	Safety-critical systems	Evolving, collaborative

1.4.2 Methodology Adopted: Scrum

Scrum was selected for this internship project because it supports adaptive planning, incremental delivery, and regular validation. The application requirements evolved during the internship, and the supervisors were able to review deliverables at the end of each sprint.

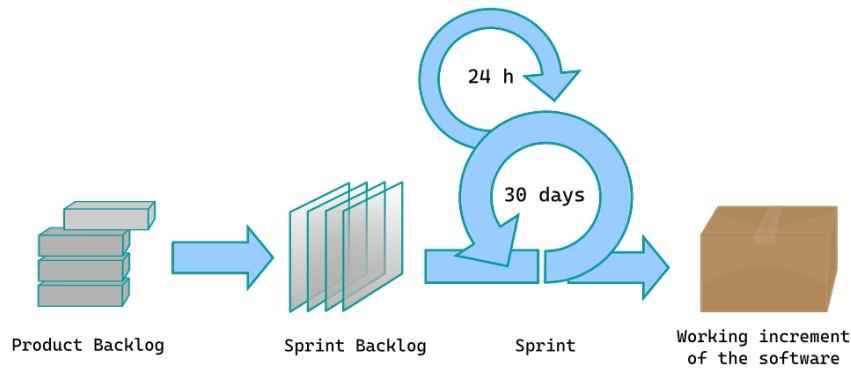


Figure 1.7: Scrum process overview

1.4.3 Scrum Process Roles

The Scrum framework defines clear roles that were adopted for this project:

- **Product Owner:** Owns the product vision, sets priorities, and manages the backlog.
- **Scrum Master:** Facilitates the process, removes impediments, and ensures Scrum practices are followed.
- **Development Team:** Builds the product increment, implements features, and delivers functionality.

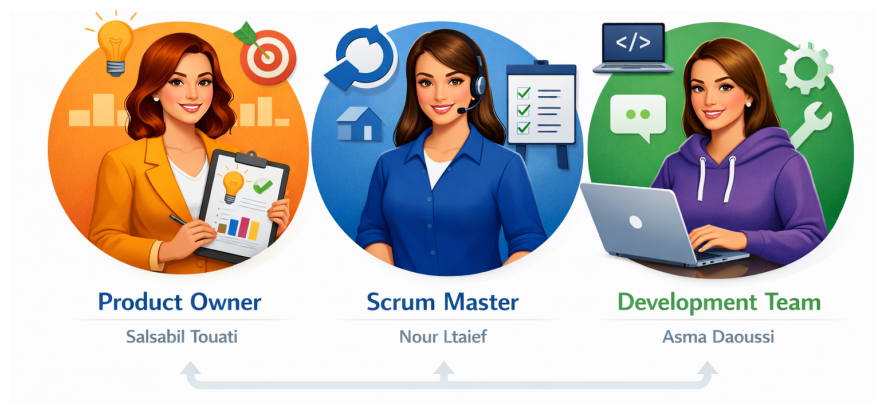


Figure 1.8: Scrum Team Roles

1.4.4 Implementation of the Scrum Methodology

The Scrum methodology was implemented using sprint planning, backlog refinement, and sprint reviews. Tasks were tracked and prioritized using ClickUp.

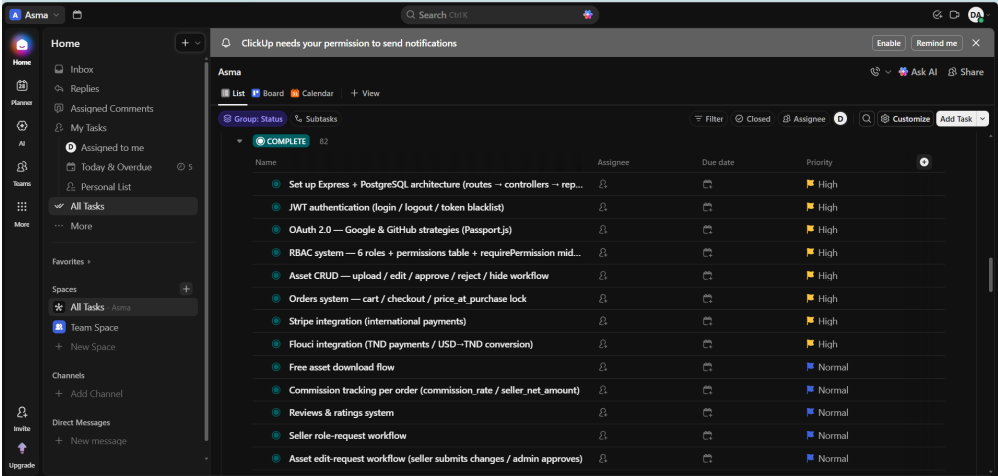


Figure 1.9: ClickUp task board

1.5 Planning

Project planning was driven by sprint sequencing, milestone delivery, and scope tracking. The development timeline spans eighteen weeks across five sprints, with each sprint delivering a coherent slice of functionality.

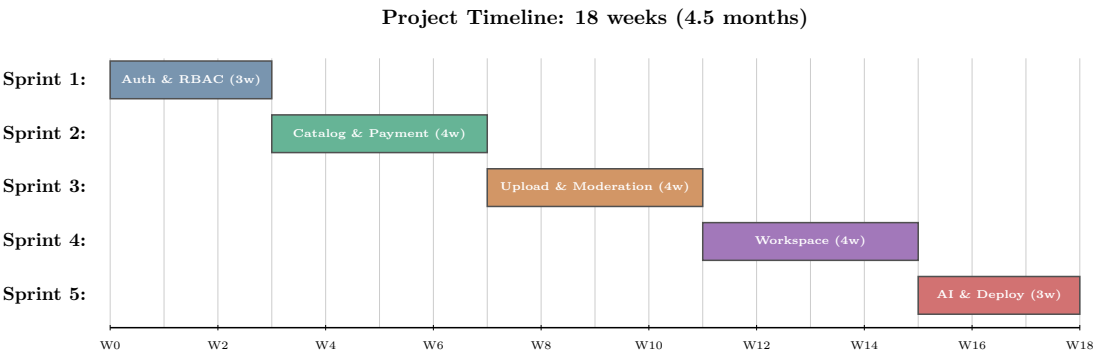


Figure 1.10: Project Timeline — 18-week Gantt chart across five Scrum sprints

Conclusion

This chapter established the project context by presenting the host organization IGS and its operational challenges, as well as a comparative analysis confirming the absence of a dedicated 3D marketplace in Tunisia. It also described the proposed two-application solution and the Scrum methodology applied across five sprints. The fol-

lowing chapter details the functional and non-functional requirements of the system, the global UML artefacts, the architectural choices and the development environment.

Requirements Analysis

Sommaire

2.1	Actors Identification	14
2.2	Needs Identification	14
2.2.1	Functional Requirements	15
2.3	Non-Functional Requirements	17
2.4	Project Management with Scrum	17
2.4.1	Product Backlog	17
2.5	Project Architecture: 3-Tier	19
2.6	Design Patterns	20
2.7	Work Environment	21
2.7.1	Hardware Environment	21
2.7.2	Software Environment	21
2.7.3	Frameworks and Libraries	23
2.7.4	Programming Languages	24
2.7.5	Data Persistence and Cloud Services	24
2.8	Global Use Case Diagrams	26
2.8.1	Marketplace — Use Case Diagram	26
2.8.2	Workspace — Use Case Diagram	26
2.9	Global Class Diagrams	27
2.9.1	Marketplace — Class Diagram	27
2.9.2	Workspace — Class Diagram	28

Introduction

This chapter defines the functional scope of the IGS Marketplace platform. It identifies the system actors, formalizes functional and non-functional requirements, and organizes the work into a product backlog. It also presents the logical architecture, the design patterns, and global use case and class diagrams for both applications.

2.1 Actors Identification

The platform is used by external marketplace users and by internal IGS collaborators. Access rights are controlled through a role-based access control model (RBAC), where each role receives a set of permissions adapted to its responsibilities.

- **Visitor**
- **Member**
- **Intern**
- **Employee**
- **Administrator**
- **Super Administrator**

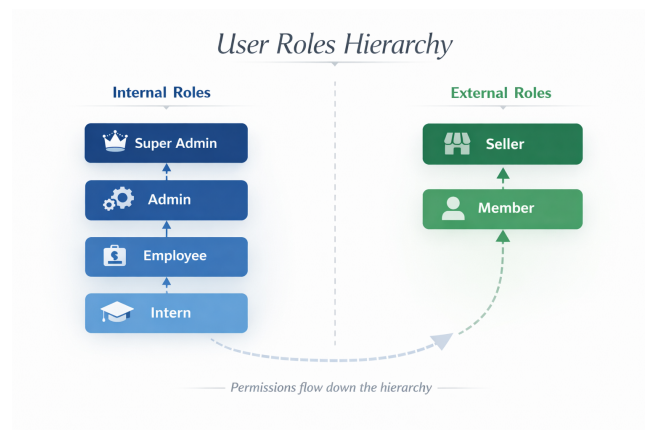


Figure 2.1: User Role Hierarchy

2.2 Needs Identification

2.2.1 Functional Requirements

The functional requirements are organized into three categories: shared requirements that apply to all users, external requirements for external users, and internal requirements for internal users, here the hierarchy is applied.

Any user can

- Login via authentication providers (email, google, github).
- Manage profile : a user can view and update their profile information, including name, email, password and profile picture.
- Browse the public catalog of assets with different categories (models, textures, animations, shades, 2d, environments).
- Search assets by name.
- Filter assets by different criteria (categories, price, tags, discount).
- Consult asset details and reviews.
- Preview assets on website or in VR.

Member can

- Create an account via email, google or github.
- Add assets to the cart, remove them, and proceed to checkout.
- Rate, review and like assets.
- View personal library.
- download free or purchased assets.
- Trade with others.
- Request seller access.
- Receive notifications

Seller

- Create an account via email, google or github.
- Upload assets.
- Communicate with the moderation team via ticketing system.

- Request edits on already approved assets.
- Consult personal asset statistics and seller analytics.

Intern

- Upload internal assets.
- Request access to paid marketplace assets with a justification.
- Redeem vouchers generated by authorized reviewers.
- Collaborate through internal projects, tasks, documents, dashboards, and real-time chat.

Employee

- Upload internal assets.
- Manage intern requests by approving or rejecting them based on the provided justification.
- Collaborate through internal projects, tasks, documents, dashboards, and real-time chat.

Administrator

- Upload internal assets.
- Manage users, roles, permissions and role requests.
- Moderate submitted assets, edit requests, intern uploads, and intern asset requests.
- Manage vouchers, categories, commission configuration, and platform settings.
- Consult platform analytics, audit logs, orders and notifications.

Super Administrator

- Upload internal assets.
- view audit logs.
- Manage platform settings.
- manage users roles.

2.3 Non-Functional Requirements

Table 2.1: Non-Functional Requirements

Category	Requirements
Security	JWT authentication, hashed passwords, OAuth support, RBAC middleware, protected internal routes, rate limiting, Helmet, CORS rules, audit logging for sensitive actions.
Performance	Indexed database queries, paginated listings, cloud storage for heavy files, client-side loading states, and optimized asset previews.
Reliability	Centralized error handling, transactional order creation, persistent notifications, and controlled moderation state transitions.
Scalability	Stateless REST API, separated React applications, PostgreSQL persistence, Supabase storage, and modular route/controller/repository layers.
Usability	Responsive marketplace screens, clear protected-route feedback, accessible forms, preview components for 3D assets, textures, HDRI, shaders, and media.
Maintainability	Layered backend architecture, reusable React components, isolated services, repository-based SQL access, and explicit database migrations/schema files.

2.4 Project Management with Scrum

The project scope is organized as a product backlog of thirty items. Each backlog item represents a user story or technical capability, prioritized by importance and assigned to one of five sprints.

2.4.1 Product Backlog

Table 2.2: Product Backlog — IGS Marketplace

ID	Feature	User Story	Sprint	Priority
1	Authentication	As a user, I can register, sign in, sign out and recover my account securely.	S1	High

ID	Feature	User Story	Sprint	Priority
2	RBAC	As an administrator, I can assign roles and permissions so each user sees only what they are allowed to.	S1	High
3	User Profile	As a user, I can view and update my personal profile information.	S1	High
4	Catalog	As a visitor, I can browse and filter the public asset catalog by category and keyword.	S2	High
5	Search & Filters	As a visitor, I can narrow results by category, price, rating, file type and availability.	S2	High
6	Asset Preview	As a buyer, I can inspect assets in 3D, VR and AR before purchasing.	S2	High
7	Cart	As a member, I can add, remove and review cart items before checkout.	S2	High
8	Payment & Orders	As a buyer, I can pay securely and receive a confirmed order record.	S2	High
9	Library & Downloads	As a buyer, I can access and download all assets I have unlocked.	S2	High
10	Seller Request	As a member, I can request seller access by providing a justification.	S3	Medium
11	Asset Upload	As a seller, I can upload files, thumbnails, metadata, tags, formats and pricing.	S3	High
12	Moderation	As an admin, I can approve, reject, hide or request changes on submitted assets.	S3	High
13	Asset Edit Request	As a seller, I can request modifications on approved assets without bypassing moderation.	S3	Medium
14	Reviews & Ratings	As a buyer, I can rate and review assets I own.	S3	Medium
15	Notifications	As a user, I can receive notifications about requests, orders and moderation events.	S3	Medium
16	Trades	As a user, I can propose and respond to peer-to-peer asset trades.	S3	Low
17	Intern Requests	As an intern, I can request access to paid assets for production needs.	S4	High
18	Vouchers	As an admin, I can issue vouchers; as a user, I can redeem them.	S4	Medium
19	Administration	As an admin, I can manage users, roles, assets, categories, orders and settings.	S4	High
20	Analytics & Audit	As an admin, I can consult analytics and trace actions through audit logs.	S4	Medium

ID	Feature	User Story	Sprint	Priority
21	Workspace Projects	As internal staff, I can create projects, assign members and monitor progress.	S4	High
22	Workspace Tasks	As internal staff, I can create, assign, update and track tasks.	S4	High
23	Workspace Chat	As internal staff, I can communicate in real time via channels and direct messages.	S4	Medium
24	AI Review Analysis	As an admin, I can use sentiment labels to support moderation decisions.	S5	Medium
25	Recommendations	As a user, I can receive personalized asset recommendations.	S5	Medium
26	Chatbot	As any user, I can ask marketplace questions through the AI assistant.	S5	Medium

2.5 Project Architecture: 3-Tier

The project follows a **three-tier architecture**:

Three-Tier Architecture, also known as three-layer architecture, organizes applications into three logical and physical layers: the presentation layer, the business logic layer, and the data access layer. This separation allows each layer to be developed, maintained, and scaled independently, improving flexibility and reducing the risk of system-wide issues when changes are made.

- **Presentation Layer (UI Layer):** Handles the user interface and interaction, displaying data and capturing user input.
- **Business Logic Layer (Application Layer):** The core of the application where rules, workflows, and data processing occur.
- **Data Access Layer (Persistence Layer):** Manages storage, retrieval, and management of data, abstracting databases and external APIs from upper layers.

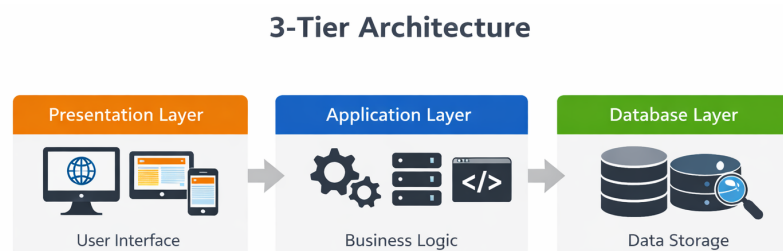


Figure 2.2: Three-Tier Architecture

2.6 Design Patterns

Five core design patterns were applied across the codebase for modularity, testability and maintainability:

Pattern	Purpose	
Three-Tier Architecture	Separates presentation, business logic and data layers so changes in one layer do not ripple through the others.	
Repository Pattern	Encapsulates all SQL queries in dedicated repository modules; controllers never write SQL directly.	
Service Layer	Centralises external integrations (Supabase, Hugging Face, Stripe) so controllers remain agnostic of provider details.	
Provider Pattern	Shares state globally across React components without prop drilling; context subscribers receive updates automatically.	
Middleware Chain	Composable middleware functions run before controllers; the chain stops at the first failure, enforcing authentication and authorization.	

2.7 Work Environment

2.7.1 Hardware Environment

The application was developed and tested on a personal laptop with the following specifications:

- **Processor:** Intel Core i7 (8th generation)
- **RAM:** 16 GB DDR4
- **Storage:** 512 GB SSD
- **Operating System:** Windows 11 Pro

For the virtual and augmented reality testing, the following headset was used:



- **Meta Quest 3** [15] A standalone mixed-reality headset manufactured by Meta Platforms, featuring full-colour passthrough cameras for augmented reality and a high-resolution display for virtual reality. It was used throughout the project to validate the WebXR immersive preview feature of the marketplace directly on the target hardware.

2.7.2 Software Environment

The development relied on a carefully selected set of open-source and cloud-based tools spanning the entire delivery chain: code editing, version control, automated integration, database management, design and documentation.



- **Visual Studio Code** [7] A lightweight, open-source source-code editor developed by Microsoft, available on Windows, macOS and Linux. It was the primary development environment for both the frontend and backend codebases, extended with ESLint, Prettier and REST-client plugins.



- **GitHub** [8] A cloud-based platform for Git version control and collaborative software development. The entire project source code was hosted on a private GitHub repository, enabling branch-based development, pull requests and automated CI/CD pipelines.



- **GitHub Actions** [9] A continuous integration and continuous delivery platform integrated into GitHub. It was used to automate linting, integration testing and the deployment pipeline triggered on every push to the main branch.



- **Docker** [10] An open-source containerization platform that packages an application and all its dependencies into a portable image. Docker was used to provide a consistent local development environment and to prepare the backend for reproducible deployment on the production server.



- **Hostinger** [11] A cloud hosting provider offering Virtual Private Server infrastructure. The backend API and the built frontend assets were deployed on a Hostinger VPS managed by Nginx as a reverse proxy and PM2 as a process supervisor.

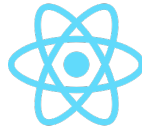


- **draw.io** [12] A free, browser-based diagramming tool used to produce the UML diagrams, architecture views and flowcharts included in this report.

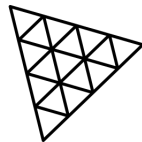


- **ESLint** [13] A static analysis tool for JavaScript that enforces code-style rules and catches common errors before runtime. It was configured with a shared rule-set across both frontends and the backend.

2.7.3 Frameworks and Libraries



- **React** [17] A declarative, component-based JavaScript library for building user interfaces, developed and maintained by Meta. Both the public marketplace and the internal workspace were built as independent React single-page applications.



- **Three.js + Model Viewer** [27, 26] A WebGL-based 3D rendering library and its companion web component. Together they power the interactive asset previews, the WebXR virtual reality mode and the augmented reality inspection feature available to Meta Quest users.



- **Socket.IO** [25] A bidirectional, event-based communication library built on top of WebSockets. It was used to implement the real-time team chat and the live notification feed shared between the marketplace and the workspace.



- **Express.js** [20] A minimal and flexible Node.js web framework chosen to expose the REST API and compose the security middleware chain covering authentication, authorization, rate limiting and input validation.



- **Tailwind CSS** [14] A utility-first CSS framework that provides low-level design tokens directly as class names. It was applied consistently across both interfaces for responsive layout and component styling.

2.7.4 Programming Languages

- **JavaScript (ES2022)** — Primary language across the entire stack: React component logic on the client, and Express controllers, repositories and services on the server [19].
- **JSX** — React’s syntactic extension that allows HTML-like markup to be written directly inside JavaScript source files to describe component trees.
- **HTML5 & CSS3** — Structural and visual foundation of both single-page applications, largely abstracted by Tailwind utility classes.
- **SQL (PostgreSQL dialect)** [21] — Used for the full relational schema, all migration scripts and the parameterised repository queries.
- **Python** — Used exclusively in the offline training phase of the AI components: sentiment analysis and intent classification models are trained with scikit-learn and then served at runtime through the HuggingFace Inference API [28].

2.7.5 Data Persistence and Cloud Services



- **PostgreSQL** [21] An open-source object-relational database system used as the primary data store. The schema spans twenty-eight tables covering users, assets, orders, RBAC, workspace entities and AI-generated data, with all access going through parameterised repository queries.



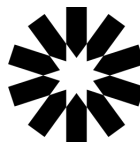
- **Supabase** [22, 23] An open-source Backend-as-a-Service platform built on PostgreSQL. It was used for cloud file storage: a public bucket for thumbnails and profile pictures, and a private bucket for three-dimensional asset files served through time-limited signed URLs.



- **Stripe** [24] An international payment processing platform used to handle card payments through the Payment Intents API. It was integrated alongside a local Tunisian gateway to support both international and dinar-denominated transactions.



- **Hugging Face** [28] A machine learning platform providing hosted inference APIs. It was used to run the DistilBERT sentiment classification model for automatic review analysis and the Qwen2.5-72B-Instruct large language model that powers the AI chatbot.



- **JSON Web Token (JWT)** [31] An open standard for compact, self-contained tokens used to transmit authentication claims between the client and the server. Every protected API route verifies the token signature and checks the in-memory revocation list before processing a request.

2.8 Global Use Case Diagrams

The project is composed of two applications sharing the same backend: the public **IGS Marketplace** and the internal **IGS Workspace**. Each one has a distinct set of actors and capabilities, so a dedicated use case diagram is presented for each.

2.8.1 Marketplace — Use Case Diagram

The Marketplace serves external users (visitors, members, sellers) and a single internal actor (intern) who can request supervised access to paid IGS-owned assets. The actor hierarchy follows the role inheritance defined in the RBAC model: a **Member** extends a **Visitor**, and a **Seller** extends a **Member**.

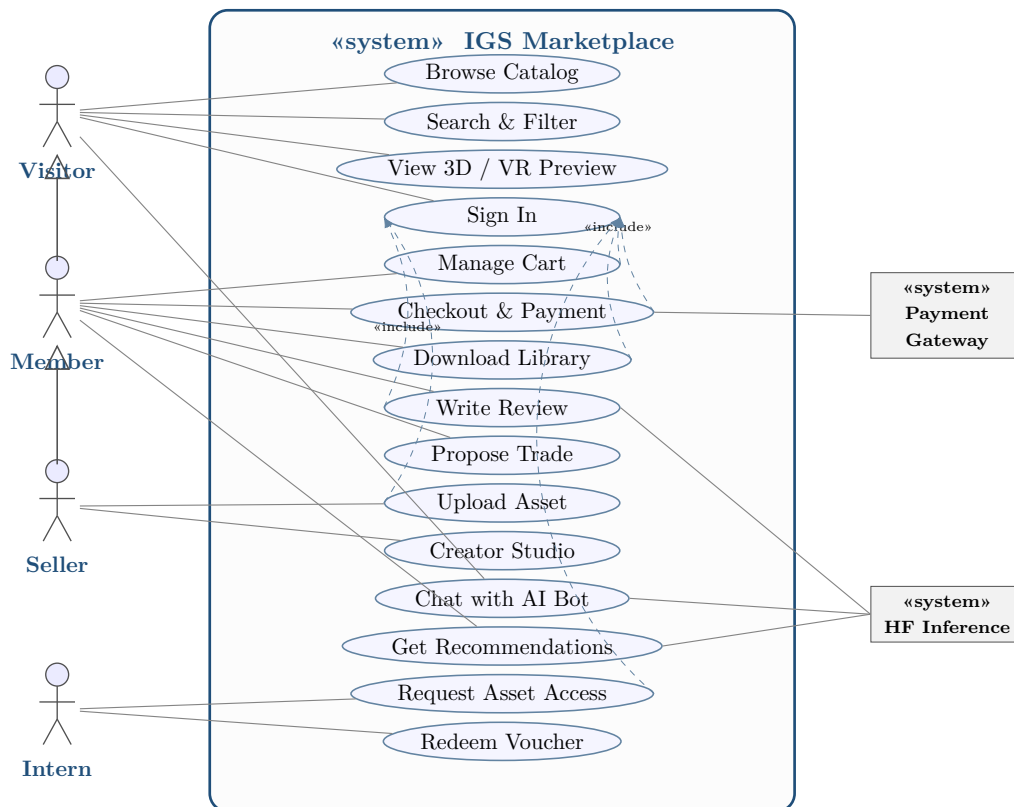


Figure 2.3: Use Case Diagram — IGS Marketplace

2.8.2 Workspace — Use Case Diagram

The Workspace is the internal ERP layer reserved for company staff. The four internal roles are stacked by hierarchy level: **Intern** (40), **Employee** (60), **Admin** (80), **Super Admin** (100). Each higher role inherits the use cases of the lower one.

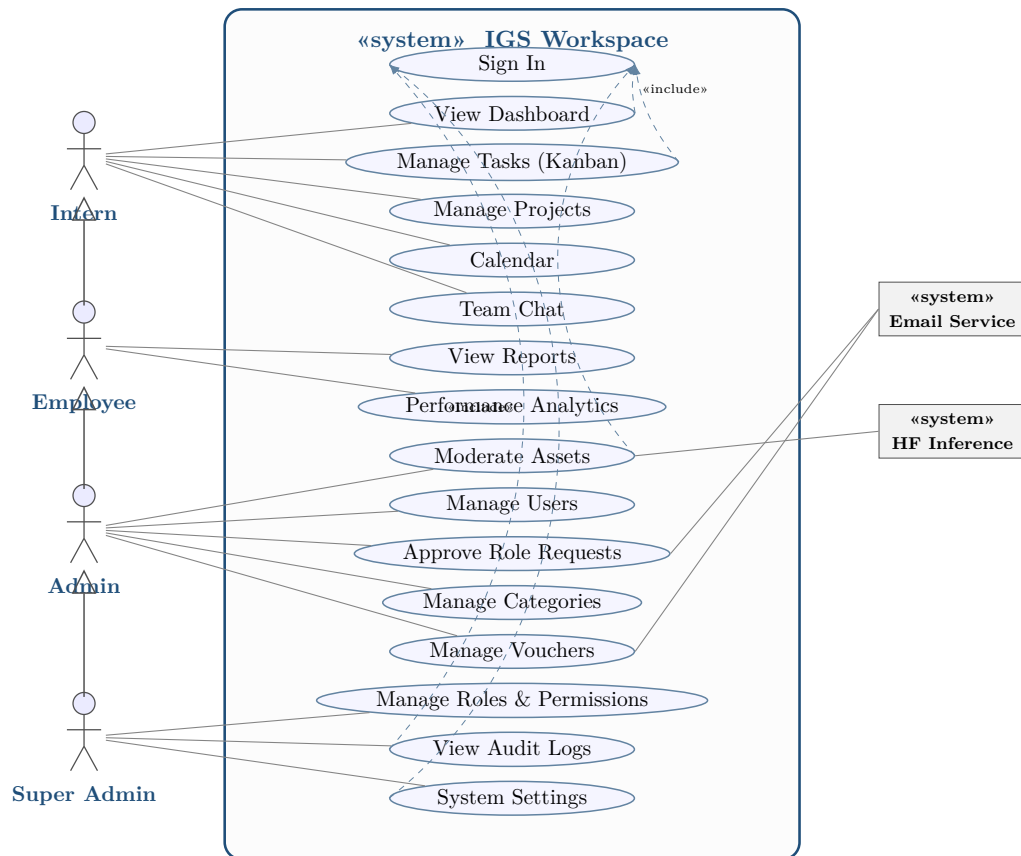


Figure 2.4: Use Case Diagram — IGS Workspace

2.9 Global Class Diagrams

Two global class diagrams are presented, one per application. They are derived from the PostgreSQL schema and the backend repositories.

2.9.1 Marketplace — Class Diagram

The Marketplace domain centres on the `User` entity (acting as both buyer and seller), connected to `Asset`, `Order`, `OrderItem`, `CartItem`, `Review`, `Trade` and `InternAssetRequest`.

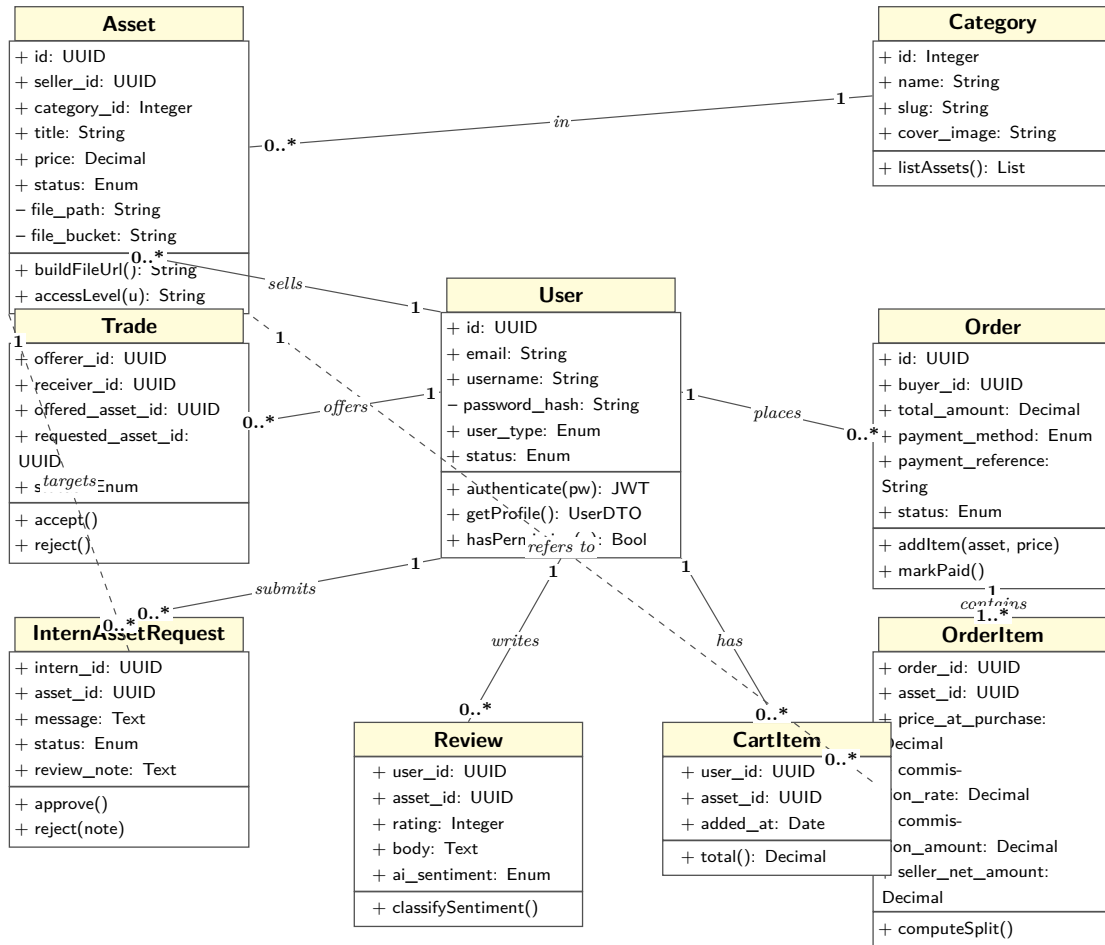


Figure 2.5: Class Diagram — IGS Marketplace

2.9.2 Workspace — Class Diagram

The Workspace domain reuses the same **User** entity but adds the RBAC model (Role, Permission), the project management entities (Project, ProjectMember, WorkspaceTask), the real-time chat (ChatChannel, Message), the Notification feed and the AuditLog trail.

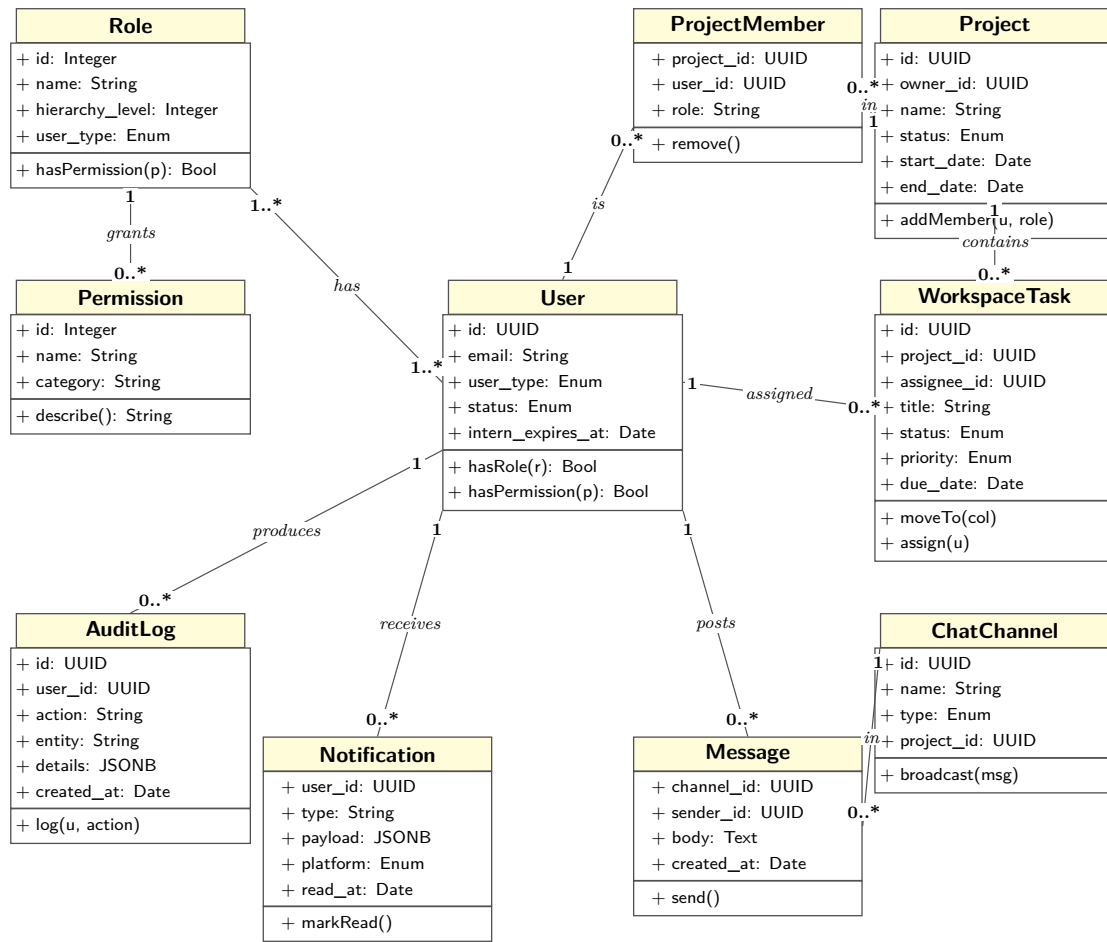


Figure 2.6: Class Diagram — IGS Workspace

Conclusion

This chapter clarified the scope of the platform, defined actors and requirements, and organized the work through a product backlog and sprint plan. It also prepared the UML artifacts needed for the rest of the report: global use case, class, and sequence views.

Marketplace Foundations

Sommaire

3.1	Sprint 1: Authentication, Profiles and RBAC	31
3.1.1	Sprint 1 Backlog	31
3.1.2	Sprint 1 Analysis	31
3.1.3	Sprint 1 Realization	35
3.1.4	Sprint 1 Conclusion	37
3.2	Sprint 2: Catalog, Cart, Stripe Payment and Library	37
3.2.1	Sprint 2 Backlog	38
3.2.2	Sprint 2 Analysis	38
3.2.3	Sprint 2 Realization	43
3.2.4	Sprint 2 Conclusion	46

Introduction

This chapter covers the first two sprints of the project. Sprint 1 establishes the security foundation: registration, sign-in, JWT issuance, OAuth callbacks and the full RBAC layer, so that every subsequent feature can rely on a role-aware user context. Sprint 2 delivers the complete public marketplace experience: a filterable asset catalog, interactive 3D and WebXR immersive previews, a shopping cart, Stripe and Flouci payment flows, and a personal download library protected by time-limited signed URLs. For each sprint the chapter presents the scoped backlog, conception diagrams (use case, sequence, class), realization screenshots and a sprint conclusion.

3.1 Sprint 1: Authentication, Profiles and RBAC

3.1.1 Sprint 1 Backlog

Table 3.1: Sprint 1 Backlog

ID	User Story	Estimation
1	As a visitor, I want to create an account using email/password or OAuth so I can access protected features.	4 days
2	As a registered user, I want to sign in and receive a JWT so I can call protected endpoints.	3 days
3	As a user, I want to reset my password by email so I can recover my account.	2 days
4	As a user, I want to manage my profile (avatar, bio, contact) so my public page stays up to date.	2 days
5	As an administrator, I want each user to belong to a role that grants a specific set of permissions.	3 days
6	As an administrator, I want sensitive actions (downloads, role changes, moderation) to be logged.	2 days

3.1.2 Sprint 1 Analysis

Use Case Diagram

The actors involved in Sprint 1 are the *Visitor*, the *Member* (any authenticated user) and the *Administrator*. The use cases gravitate around the authentication ceremony:

Register, Sign In, OAuth Login, Password Reset, Manage Profile, Assign Role and Audit Action.

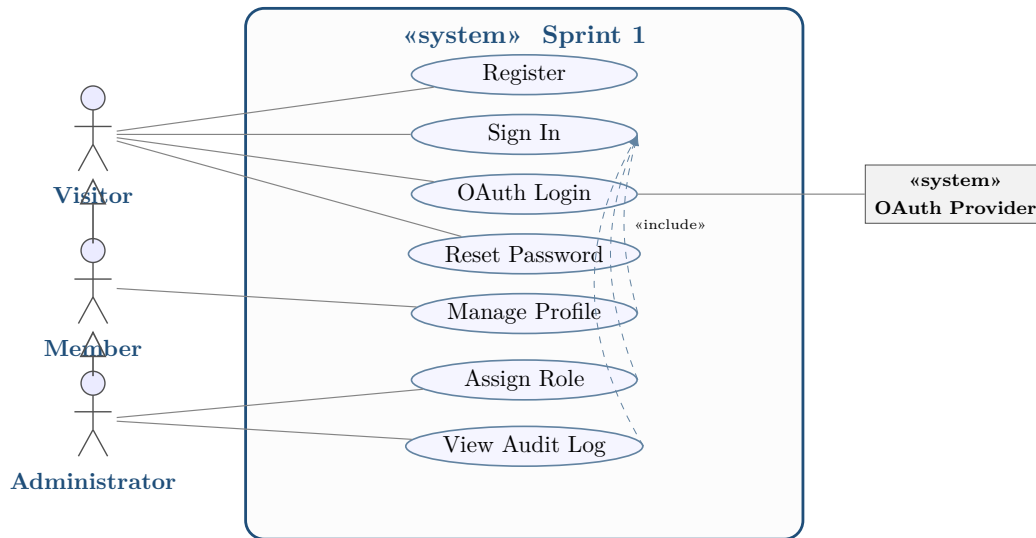


Figure 3.1: Use case diagram — Sprint 1

Use Case Description — UC-01: Register and Authenticate

Table 3.2: Use Case Description: Register and Authenticate

UC-01 — Register and Authenticate	
Use case	Register and Authenticate
Actor	Visitor / Registered User
Precondition	The registration form is accessible. For sign-in, the user already has an account.
Postcondition	The user is authenticated, holds a valid JWT, and can access protected routes.
Nominal scenario	1. The visitor fills in username, email, and password on the registration page. 2. The system validates the input and hashes the password with bcrypt. 3. A user record is created with the <i>member</i> role and status <i>active</i>. 4. The system issues a signed JWT and redirects the user to the marketplace. 5. On subsequent visits, the user enters credentials and receives a fresh JWT.
Alternative scenario	A1 OAuth login: The user authenticates via Google or GitHub; the system creates or updates the account and issues a JWT. A2 Forgotten password: The user requests a reset link by email and sets a new password through the reset form. E1 Invalid credentials: The system returns HTTP 401; the token is not issued.

Sequence Diagram — Authentication

The sign-in flow walks through the `authController`, the `users` table and the JWT signing service. After successful credential validation, the controller returns a signed JWT that the browser stores in `localStorage`. Every subsequent call passes this token through the `authenticate` middleware.

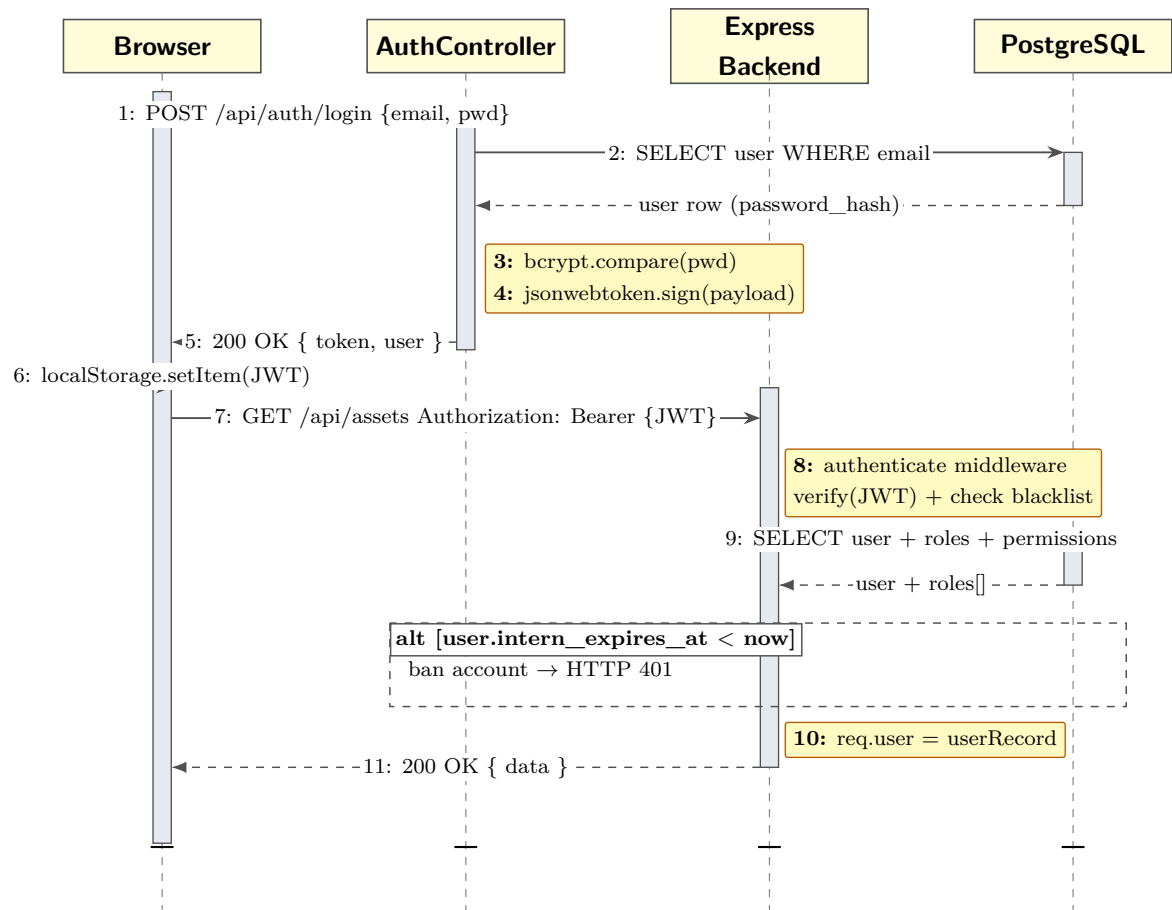


Figure 3.2: Sequence diagram — Authentication flow

Class Diagram

The Sprint 1 domain centres on the **User** entity linked to the RBAC trio **Role**, **Permission** and **AuditLog**. A user holds multiple roles; each role carries a set of permissions; every sensitive action is traced in the audit log.

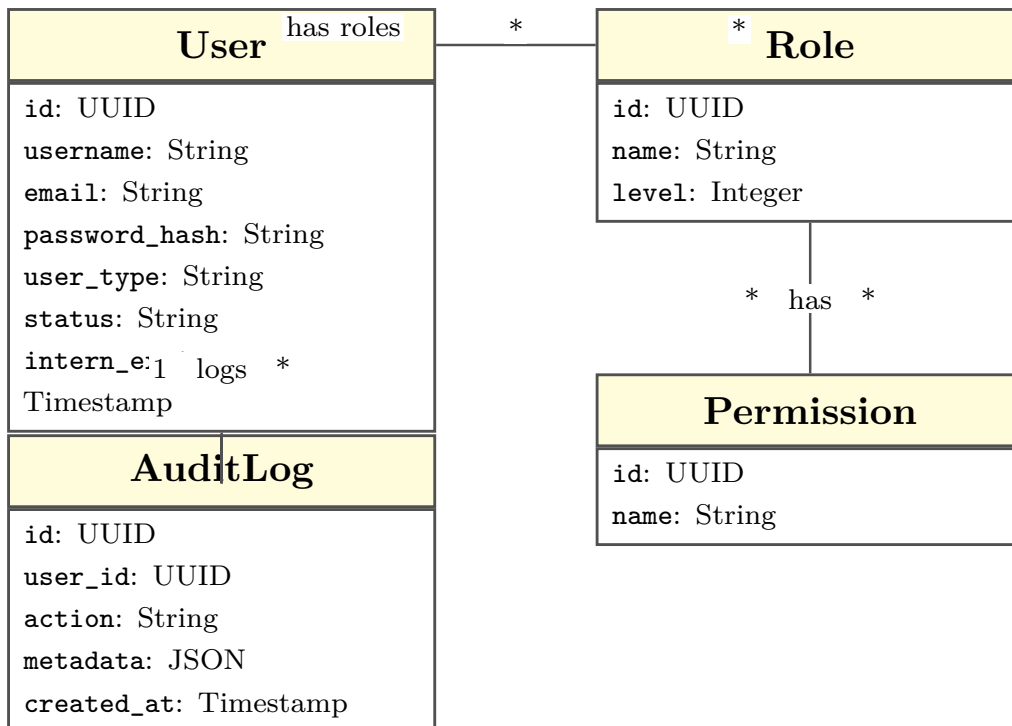


Figure 3.3: Class diagram — Sprint 1: Authentication & RBAC

3.1.3 Sprint 1 Realization

Authentication flow. Authentication is the entry point of any action that goes beyond public browsing. The system supports three login methods: email/password, Google OAuth and GitHub OAuth. All three paths ultimately produce a JWT stored in `localStorage['igs_token']`. The token is verified on every protected API call by the `authenticate` middleware, which attaches a `req.user` object containing the user's ID, email, roles, permissions and intern expiry date.

After a successful `POST /api/auth/login`, the backend:

1. Verifies the password hash with `bcrypt.compare`.
2. Checks the user's `status` (active / suspended).
3. Signs a JWT with `jsonwebtoken` using `JWT_SECRET`.
4. Returns `{ token, user }` to the frontend.

Logout is enforced by adding the token to an in-memory blacklist (`services/tokenBlacklist.js`). On every subsequent request the `authenticate` middleware extracts the Bearer token, checks the blacklist, verifies the signature, then queries the DB for the full user record including roles and permissions. For intern accounts, it auto-bans the user if `intern_expires_at` is in the past.

OAuth integration. OAuth flows use Passport.js strategies. After the provider redirects back to `/api/auth/callback/:provider`, the backend upserts the user record and issues a JWT, then redirects to the frontend with the token in the query string. The React callback page (`Pages/Auth/AuthCallback.jsx`) reads the token, stores it and navigates to the intended page.

Role hierarchy. Six roles govern what users can see and do on the platform. Each role has a numeric level used for hierarchical comparisons.

Table 3.3: Role hierarchy and access levels

Role	User Type	Level	Key capabilities
super_admin	internal	100	Full platform control, system config
admin	internal	80	User, asset and order management
employee	internal	60	Content moderation, analytics
intern	internal	40	Workspace access, supervised uploads
seller	external	20	Asset uploads and sales stats
member	external	10	Purchase, cart, library, reviews

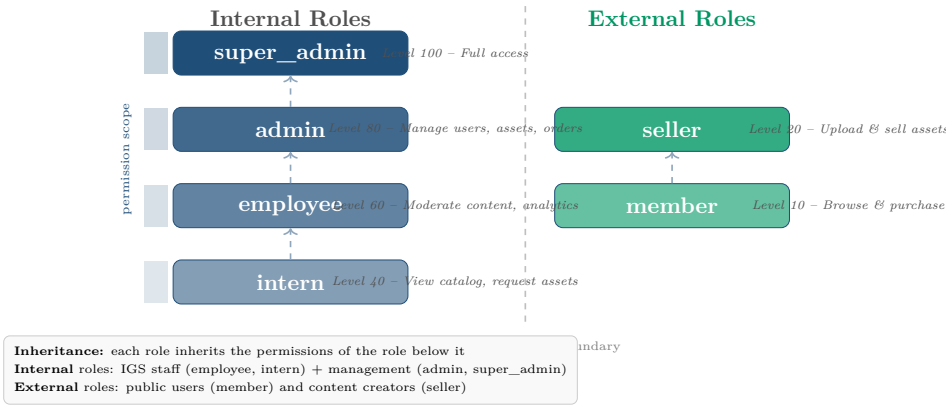


Figure 3.4: RBAC role and permission hierarchy

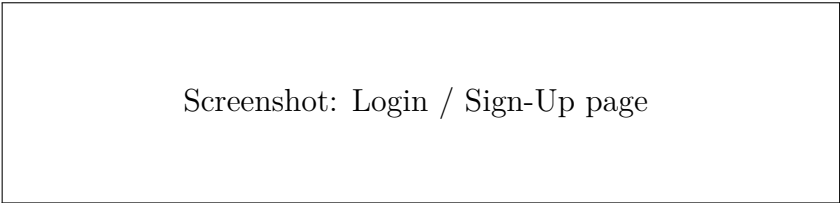


Figure 3.5: Login and Sign-Up interface (placeholder)



Figure 3.6: Profile management interface (placeholder)

Interfaces (Sprint 1).

3.1.4 Sprint 1 Conclusion

At the end of Sprint 1 the platform has a working JWT-based authentication, OAuth support, a hierarchical RBAC model with six roles and an audit log capturing sensitive actions. This foundation conditions every subsequent feature delivered in the following sprints.

3.2 Sprint 2: Catalog, Cart, Stripe Payment and Library

3.2.1 Sprint 2 Backlog

Table 3.4: Sprint 2 Backlog

ID	User Story	Estimation
1	As a visitor, I want to browse a paginated catalogue with filters by category, price and tags.	3 days
2	As a visitor, I want to consult an asset detail page with a 3D preview adapted to the asset type.	4 days
3	As a member, I want to enter VR / immersive mode for compatible 3D assets so I can inspect them at scale.	2 days
4	As a member, I want to add or remove assets from a personal cart.	2 days
5	As a member, I want to pay for the cart through Stripe and receive instant access.	4 days
6	As a member, I want a personal library listing all the assets I can download, with signed-URL downloads.	3 days
7	As an intern, I want to request supervised access to paid IGS-owned assets I cannot purchase myself.	2 days

3.2.2 Sprint 2 Analysis

Use Case Diagram

Sprint 2 actors are the *Visitor*, *Member*, *Intern* and the external *Stripe payment gateway*. The use cases cover browsing, previewing, cart management, Stripe checkout, library downloads and intern access requests.

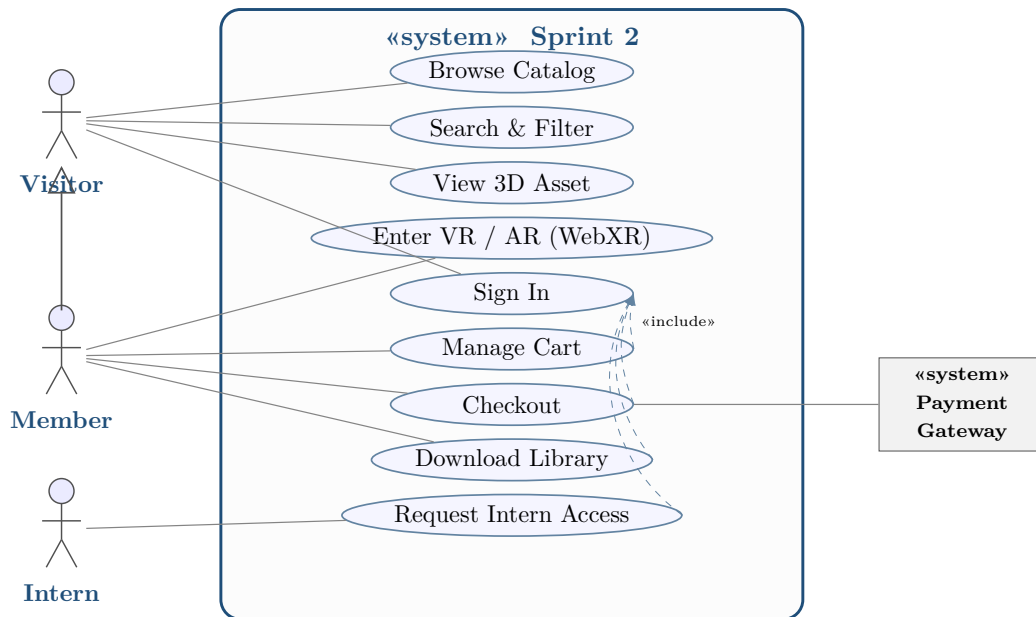


Figure 3.7: Use case diagram — Sprint 2

Use Case Description — UC-02: Purchase an Asset

Table 3.5: Use Case Description: Purchase an Asset

UC-02 — Purchase an Asset	
Use case	Purchase an Asset
Actor	Member
Precondition	The member is authenticated. At least one approved paid asset exists in the catalog.
Postcondition	The order is recorded as <i>paid</i> and the asset is available in the member's library.
Nominal scenario	1. The member browses the catalog, opens an asset detail page, and inspects the 3D preview. 2. The access-level check returns <i>purchasable</i>; the member adds the asset to the cart. 3. The member proceeds to checkout and selects a payment method (Stripe or Flouci). 4. The payment provider confirms the transaction; the backend creates a paid order. 5. The asset is unlocked in the member's library and a signed download URL is generated.
Alternative scenario	A1 Free asset: Access level returns <i>free_download</i>; the asset is added directly to the library without checkout. A2 Already owned: Access level returns <i>owned</i>; the download button is shown without re-payment. E1 Payment failure: The order stays <i>pending</i> and library access is not granted.

Sequence Diagram — Stripe Payment

The Stripe payment sequence orchestrates the frontend cart, the Express backend, the Stripe API and the Supabase storage layer. After Stripe confirms the payment intent, the backend creates the order, copies cart items into `order_items` with the locked commission split, and grants library access.

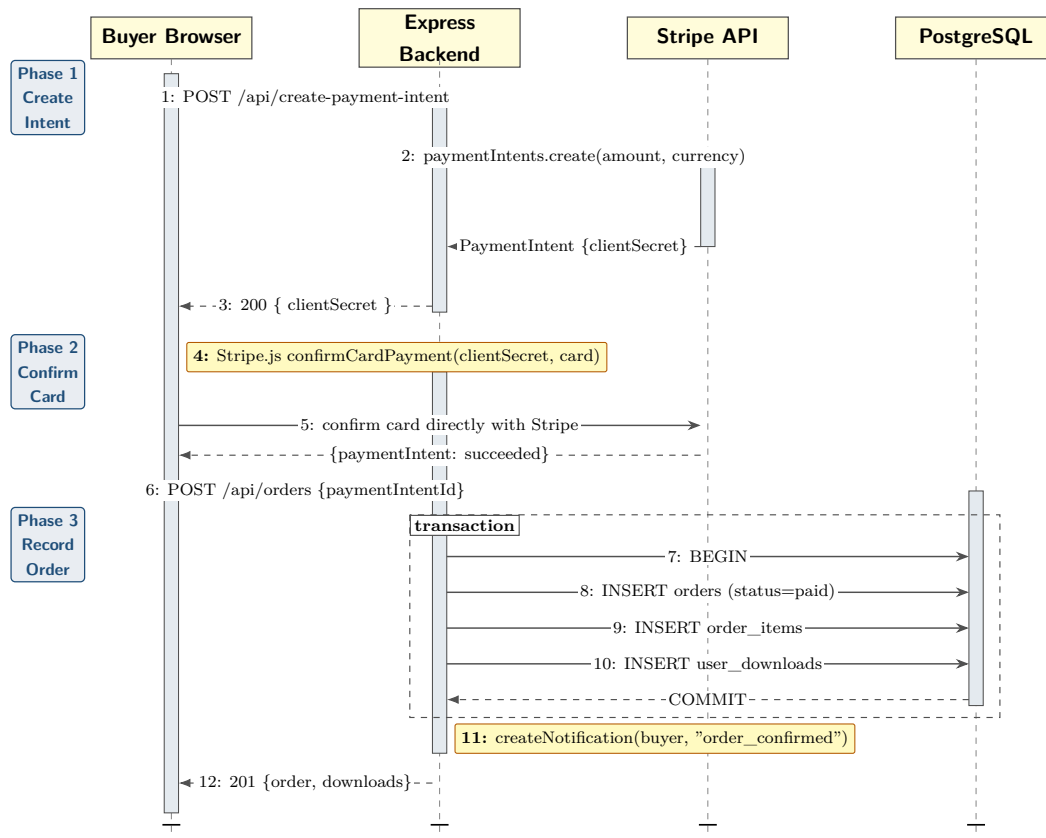


Figure 3.8: Sequence diagram — Stripe payment to library unlock

Sequence Diagram — Intern Access Request

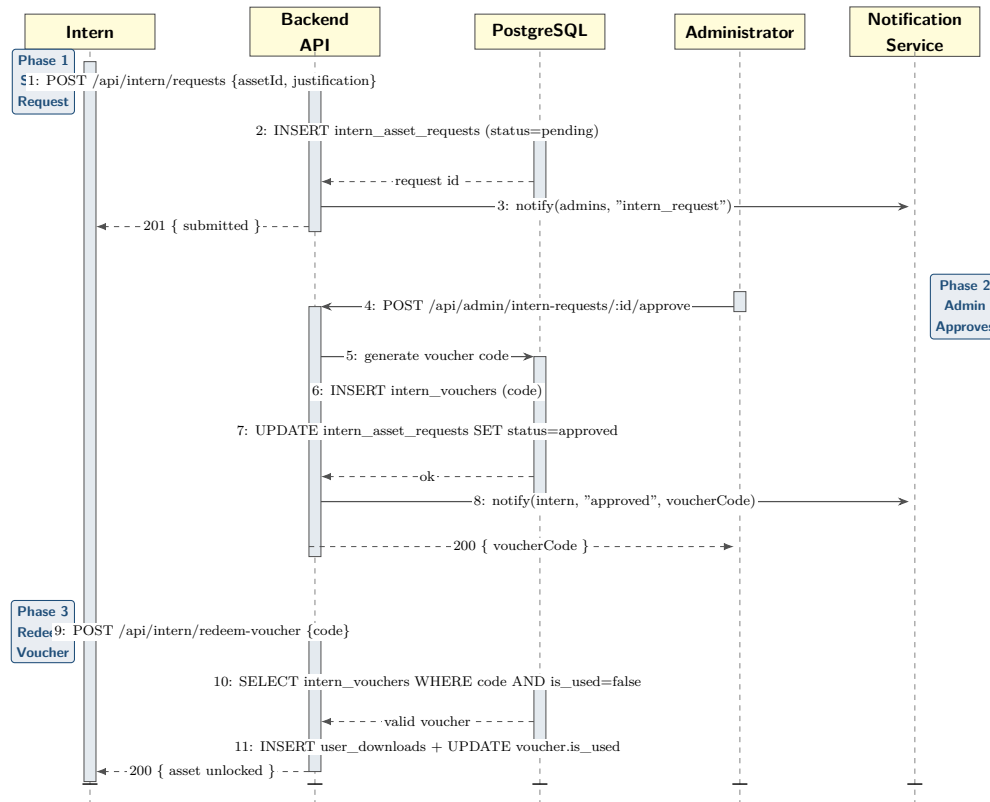


Figure 3.9: Sequence diagram — Intern asset request

Class Diagram

The Sprint 2 domain introduces the **Asset** and **Category** entities alongside the purchase pipeline: **CartItem**, **Order**, **OrderItem** and **UserDownload**. Each order item locks the price and commission split at purchase time so revenue figures remain stable regardless of future price changes.

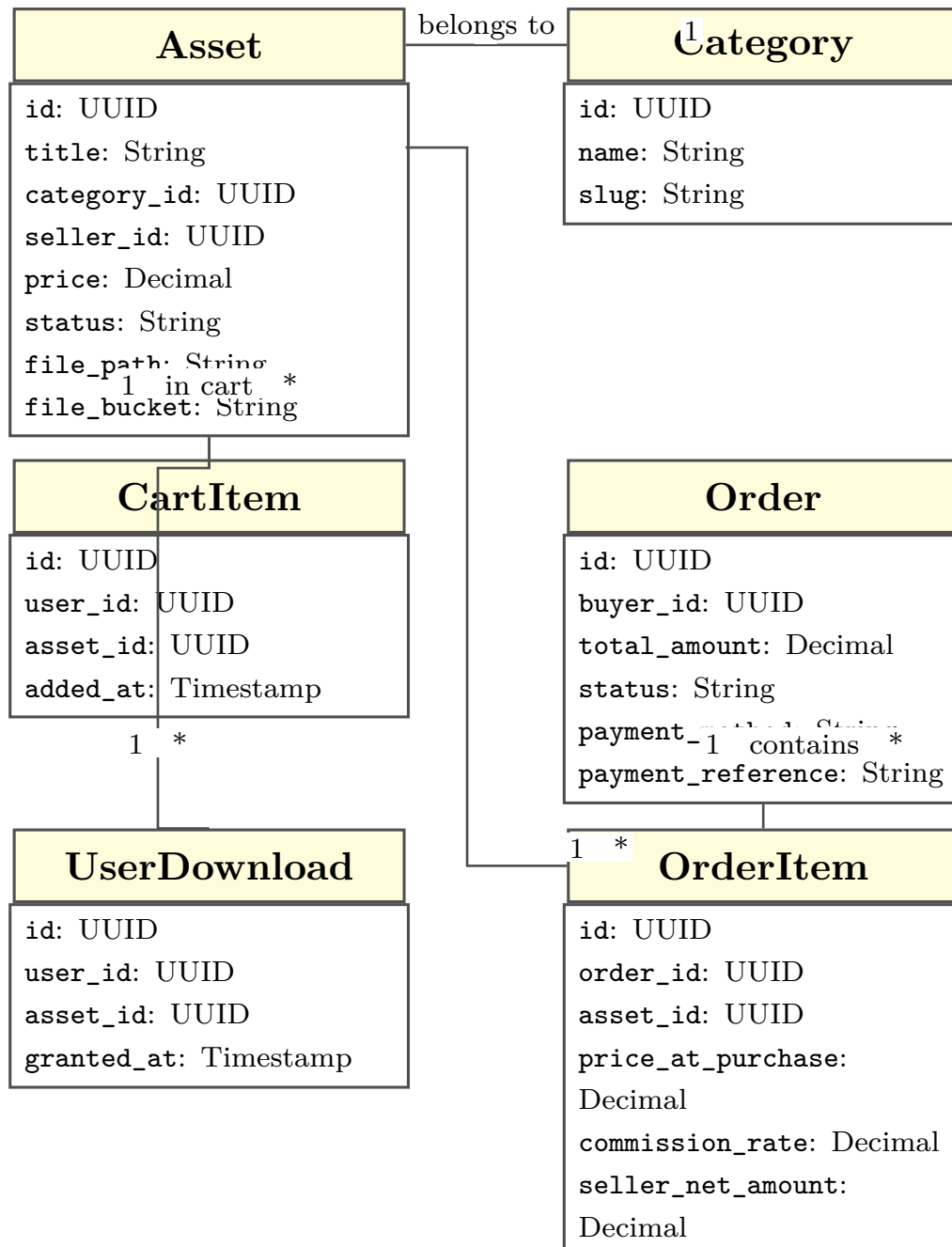


Figure 3.10: Class diagram — Sprint 2: Catalog, Cart, Payment & Library

3.2.3 Sprint 2 Realization

Categories and discovery. The marketplace supports thirteen asset categories: models, textures, HDRIs, shaders, scripts, plugins, audio, VFX, 2D art, printables, animations, environments and decals. Each category page passes a `category` prop to the shared `<ItemsListing />` component, which fetches paginated results from `GET /api/assets`. The catalog endpoint supports filtering by category, price range, rating, file type, tags and sort order.

Asset detail page and 3D viewer. The asset detail page (`Pages/View.jsx`) renders an interactive preview adapted to the asset type:

Table 3.6: Asset type to preview component mapping

Type	Preview component
models / printables	<code><model-viewer></code> web component or Three.js scene
textures / decals	<code>TextureSphere</code> — thumbnail mapped on a Three.js sphere
shaders	<code>ShaderPreview</code> — live GLSL rendering
scripts	<code>ScriptViewer</code> — syntax-highlighted source
hdri	<code>HdriViewer</code> — equirectangular panorama (Three.js)
audio	<code>AudioPlayer</code> with waveform visualization
animations	<code>AnimationViewer</code> — FBX skeleton animated with <code>AnimationMixer</code>
environments	YouTube embed when <code>metadata.youtube_id</code> is present

WebXR immersive preview. For compatible models, `Components/ModelViewer.jsx` enables immersive VR/AR through the WebXR Device API: `renderer.xr.enabled = true` activates stereo rendering and `VRButton/ARButton` are injected from `three/examples/jsm/webxr/`. When `renderer.xr.isPresenting` is true, the post-processing `EffectComposer` (Bloom and Bokeh passes) is bypassed because they crash in XR stereo mode.

Asset access control. For each asset the backend computes an access level string via `GET /api/assets/:id/access`. The logic implemented in `computeAssetAccess()` returns one of:

1. **owned** — already in the user’s library.
2. **free_download** — asset is free and the user is authenticated.
3. **auto_grant_igs** — IGS-owned asset, internal collaborator above intern.
4. **requires_request** — IGS-owned paid asset; intern must request approval.
5. **purchasable** — paid asset, not yet purchased.
6. **blocked** — not authenticated or insufficient role.

Payment pipeline. The platform supports two checkout paths:

Table 3.7: Payment methods

Method	Provider	Use case
Stripe	Stripe Inc. (international)	Credit/debit card payments
Free	Internal (no gateway)	Zero-price assets, instant unlock

Every paid sale records the commission split in `order_items`: `commission_rate` (platform rate read from `platform_config`, default 15%), `commission_amount` (absolute amount kept by IGS) and `seller_net_amount` (amount remitted to the seller). The Stripe webhook endpoint listens for `payment_intent.succeeded` events, updates the order status to `paid`, grants download access and triggers in-app notifications.

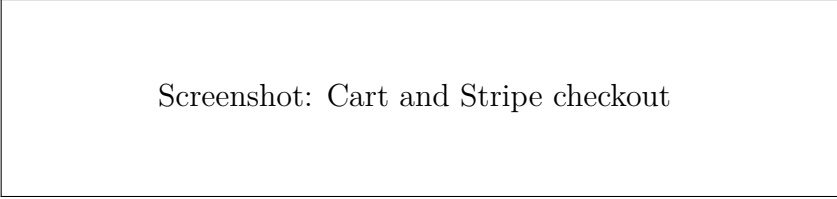
Secure file delivery. Asset files are stored in Supabase Storage across two buckets: `assets-images` (thumbnails, public, 10 MB max) and `assets-files` (3D models, ZIPs, scripts, private, 500 MB max). The `assets` table stores `file_path` and `file_bucket` rather than a public URL; the repository layer reconstructs the URL at SELECT time. When a user requests a file via `GET /api/assets/:id/download-url` the backend re-checks the access level, calls `storageService.createSignedDownloadUrl(bucket, filePath, 3600)` to generate a one-hour signed URL, and audit-logs super-admin overrides as `SUPERADMIN_OVERRIDE_DOWNLOAD`.

Screenshot: Catalog page with filters

Figure 3.11: Catalogue browsing interface (placeholder)

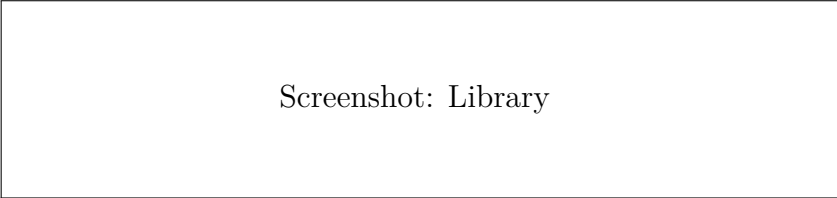
Screenshot: Asset detail with 3D viewer

Figure 3.12: Asset detail page with interactive 3D preview (placeholder)



Screenshot: Cart and Stripe checkout

Figure 3.13: Cart and Stripe checkout (placeholder)



Screenshot: Library

Figure 3.14: Personal download library (placeholder)

Interfaces (Sprint 2).

3.2.4 Sprint 2 Conclusion

At the end of Sprint 2 the public marketplace is fully functional: visitors can browse and preview assets in 3D, members can purchase through Stripe, the library exposes signed downloads with one-hour TTL and interns can request supervised access to IGS-owned paid assets.

Conclusion

This chapter delivered the authentication backbone and the commercial heart of the marketplace. Sprint 1 ensured every subsequent request is authenticated and role-controlled; Sprint 2 turned the platform into a usable storefront with a rich 3D preview, a Stripe-powered checkout and a secure library. The next chapter covers Sprint 3 (seller workflow: upload, moderation, reviews, Creator Studio, trades) and Sprint 4 (the internal IGS Workspace). AI-driven features are presented separately in Chapter 5.

Seller Workflow and Workspace

Sommaire

4.1	Sprint 3: Asset Upload, Moderation, Reviews and Seller Workflow	48
4.1.1	Sprint 3 Backlog	49
4.1.2	Sprint 3 Analysis	49
4.1.3	Sprint 3 Realization	53
4.1.4	Sprint 3 Conclusion	55
4.2	Sprint 4: IGS Workspace and Administration	56
4.2.1	Sprint 4 Backlog	56
4.2.2	Sprint 4 Analysis	56
4.2.3	Sprint 4 Realization	60
4.2.4	Sprint 4 Conclusion	63

Introduction

This chapter presents Sprint 3 and Sprint 4. Sprint 3 closes the production loop on the public marketplace: sellers request access and upload assets, administrators moderate the queue with full approve/reject/edit workflows, buyers leave ratings and written reviews, peer-to-peer trades become possible, and the Creator Studio dashboard gives each seller a full view of their revenue and downloads. Sprint 4 then delivers the internal **IGS Workspace**: project boards, task tracking, a real-time team chat, a calendar, intern asset-request management, voucher issuance and the full administrative control panel covering users, roles, orders, categories and audit logs. AI-driven features (review sentiment analysis, recommendations and the chatbot) are deliberately deferred to Sprint 5 where they are presented as a coherent block alongside security hardening and deployment.

4.1 Sprint 3: Asset Upload, Moderation, Reviews and Seller Workflow

4.1.1 Sprint 3 Backlog

Table 4.1: Sprint 3 Backlog

ID	User Story	Estimation
1	As a member, I want to request seller access with a justification so I can publish assets.	2 days
2	As a seller, I want to upload assets (file + thumbnail + metadata) and submit them for moderation.	4 days
3	As an admin, I want a moderation queue where I can approve, reject or request changes on submitted assets.	3 days
4	As a seller, I want to request edits on already-approved assets without bypassing moderation.	2 days
5	As a buyer, I want to leave a rating and a written review on owned assets.	2 days
6	As a user, I want to be notified about request, order and moderation events.	2 days
7	As a seller, I want a creator studio with stats (revenue, sales, downloads, views) and an asset table.	3 days
8	As a user, I want to propose peer-to-peer trades of assets I own against assets I want.	2 days
9	As a user, I want to view, subscribe to and cancel seller subscription plans.	2 days

4.1.2 Sprint 3 Analysis

Use Case Diagram

The Sprint 3 actors are the *Member*, *Seller* and *Admin*. The use cases cover seller onboarding, asset upload and moderation, edit requests, reviews and ratings, notifications, the creator studio dashboard, peer-to-peer trades and subscription management. AI-driven features (sentiment analysis and recommendations) are deferred to Sprint 5 along with the rest of the AI integration.

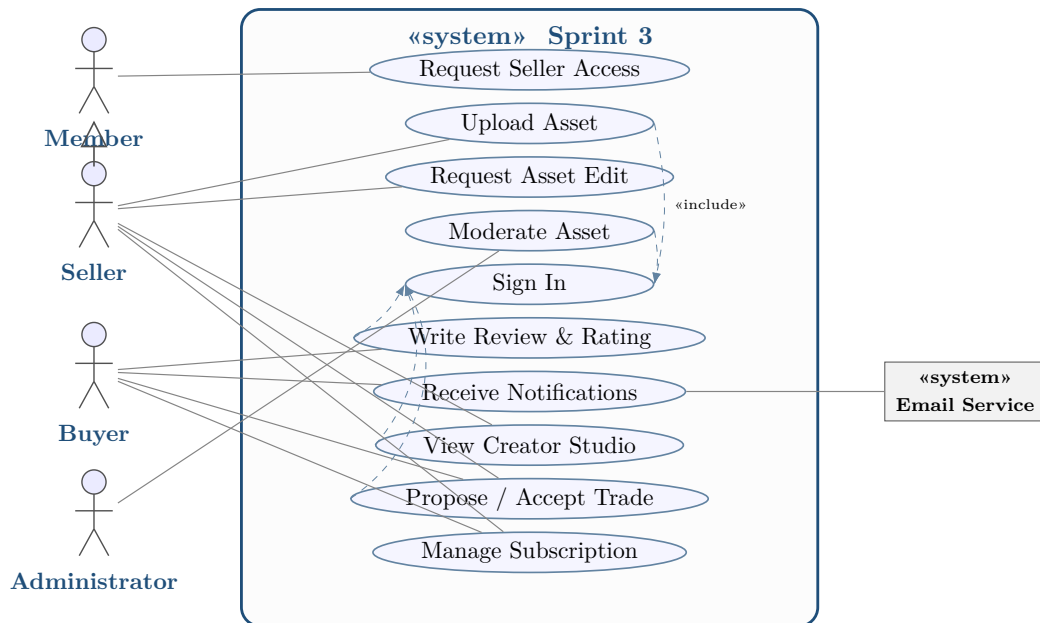


Figure 4.1: Use case diagram — Sprint 3

Use Case Description — UC-03: Upload and Submit an Asset

Table 4.2: Use Case Description: Upload and Submit an Asset

UC-03 — Upload and Submit an Asset	
Use case	Upload and Submit an Asset
Actor	Seller / Administrator
Precondition	The actor holds the <i>seller</i> role. File size is within the 500 MB limit.
Postcondition	The asset is publicly visible in the catalog with status <i>approved</i> , or returned to the seller for corrections.
Nominal scenario	1. The seller fills in the asset form (title, category, tags, price, formats). 2. The server validates the thumbnail magic bytes and stores it in the public images bucket. 3. The server validates the 3D file MIME type and stores it in the private files bucket. 4. The asset record is created with status <i>pending</i>; no public URL is generated. 5. The administrator reviews the submission in the moderation panel and approves it. 6. The asset status changes to <i>approved</i> and appears in the public catalog.
Alternative scenario	A1 Rejection: The admin rejects the submission with a note; the seller is notified and can resubmit. A2 Internal upload: When an employee or intern uploads, the seller is forced to the IGS Studio account and the uploader ID is stored separately. E1 MIME mismatch: The server rejects the file with HTTP 400 before reaching storage.

Sequence Diagram — Asset Upload

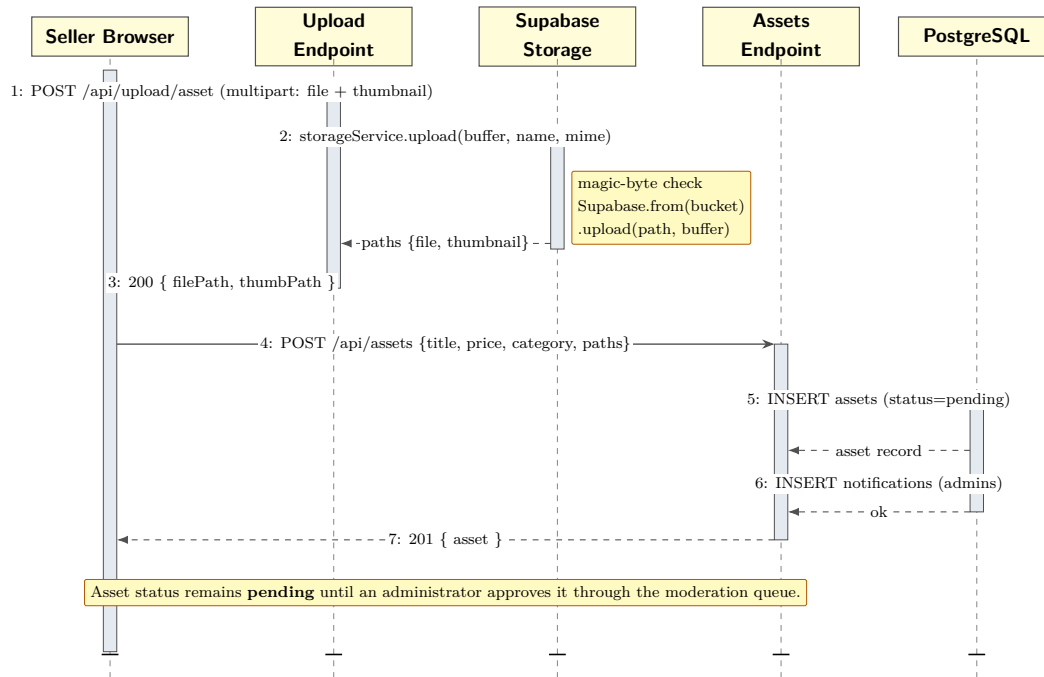


Figure 4.2: Sequence diagram — Seller upload to moderation queue

Class Diagram

Sprint 3 extends the `Asset` entity with a status state machine and connects it to `AssetEditRequest`, `Review` (with an `ai_sentiment` column reserved for Sprint 5), `Trade` and `Subscription`.

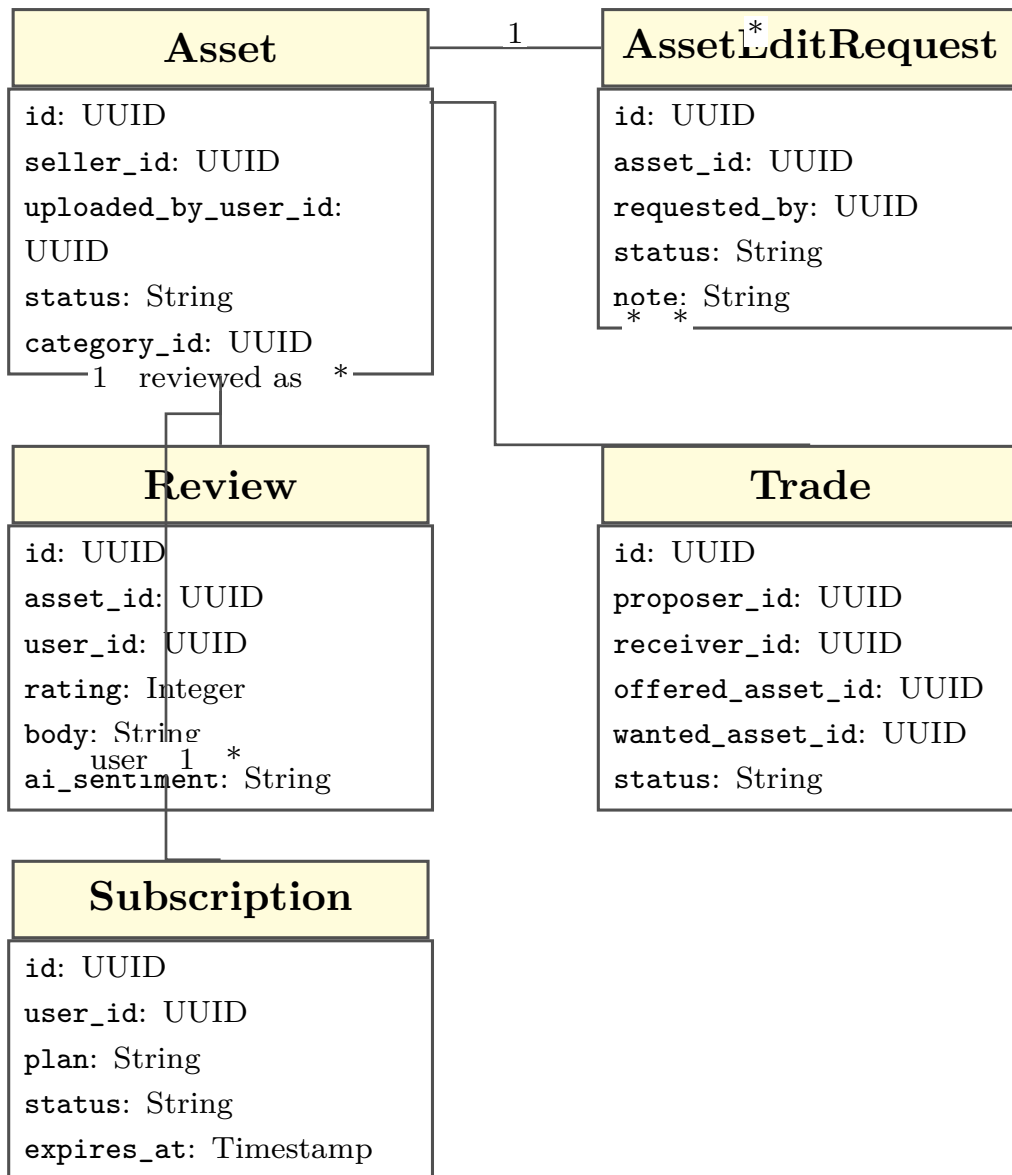


Figure 4.3: Class diagram — Sprint 3: Upload, Moderation & Reviews

4.1.3 Sprint 3 Realization

Upload flow. Files are uploaded in two stages: `POST /api/upload/image` uploads the thumbnail to `assets-images` with magic-byte validation (PNG, JPG, WEBP, GIF), then `POST /api/upload/asset` uploads the asset file to `assets-files` with category-specific MIME-type filtering. Once both uploads return storage paths, the frontend calls `POST /api/assets` with the metadata; the asset is created with status `pending` and enters the moderation queue.

When an internal user (employee, intern, admin) uploads an asset, the backend forces `seller_id = '49475300-0000-4000-8000-000000000001'` (the IGS Studio UUID) and sets `uploaded_by_user_id = req.user.id` for attribution.

Moderation queue. Administrators access `/moderation` where two tabs surface pending submissions: external sellers (status `pending`) and intern uploads (status `pending_supervisor`). Each row is expandable to show the asset metadata; the admin can attach a review note and click *Approve* or *Reject*. Decisions trigger notifications to the seller.

Reviews and ratings. Users who own an asset can leave one review composed of a star rating (`asset_ratings`) and a written body (`asset_reviews`). The endpoint `POST /api/assets/:id/reviews` verifies ownership before accepting a submission. Reviews are displayed in chronological order on the asset detail page. The `ai_sentiment` column on the review record is reserved for the automated sentiment-analysis pipeline introduced in Sprint 5.

Notifications. Each significant action (review left, request accepted, asset moderated, trade proposed) triggers a row in the `notifications` table. The frontend polls `GET /api/notifications` on app mount and refreshes through the `Socket.IO notif:new` event. A `platform` column (`marketplace`, `workspace` or `both`) controls which interface receives the entry.

Creator studio. The creator studio (`Pages/Studio.jsx`, permission `view_own_stats`) exposes six stat cards (total revenue, total sales, pending assets, active listings, total downloads, total views), a recent-sales list and a paginated asset table with status badges, counts, revenue and an *Edit* link per row. Data comes from `GET /api/seller/dashboard` and `GET /api/seller/assets`.

Trades. `Pages/Trade.jsx` lets a user offer one of their owned assets in exchange for an asset owned by another user. Each trade has a state machine (`pending` / `accepted` / `rejected`) and triggers notifications to both parties.

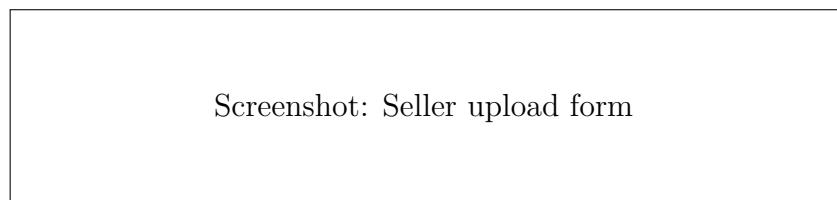


Figure 4.4: Asset upload form (placeholder)



Screenshot: Moderation queue

Figure 4.5: Moderation queue interface (placeholder)



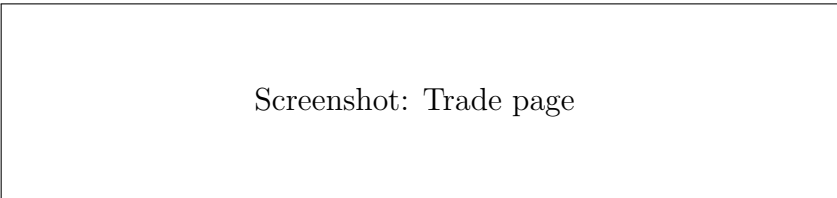
Screenshot: Review list and rating widget

Figure 4.6: Review list and rating widget (placeholder)



Screenshot: Creator Studio dashboard

Figure 4.7: Creator Studio dashboard (placeholder)



Screenshot: Trade page

Figure 4.8: Trade composer and incoming requests (placeholder)

Interfaces (Sprint 3).

4.1.4 Sprint 3 Conclusion

Sprint 3 closes the production loop on the public marketplace. Sellers can request access, publish assets and follow their performance through the creator studio dashboard. Administrators control the catalog through the moderation queue. Buyers can rate, review and propose peer-to-peer trades, while notifications keep every actor informed of state changes. The AI-driven sentiment classification and the recommendation engine that build on these reviews are introduced in Sprint 5.

4.2 Sprint 4: IGS Workspace and Administration

4.2.1 Sprint 4 Backlog

Table 4.3: Sprint 4 Backlog

ID	User Story	Estimation
1	As an internal user, I want a workspace with role-gated navigation so I see only the pages I can access.	3 days
2	As an intern, I want a Kanban task board with priorities and assignees.	3 days
3	As an internal user, I want to create and manage projects with members and a date range.	2 days
4	As an internal user, I want a calendar that shows task deadlines and project milestones.	1 day
5	As an internal user, I want a real-time chat backed by Socket.IO.	3 days
6	As an employee, I want analytics reports on revenue, sales, categories and per-team performance.	2 days
7	As an admin, I want to manage users, role requests, categories and vouchers.	3 days
8	As an admin, I want to approve / reject intern asset requests with a review note.	2 days
9	As a super-admin, I want to view audit logs filtered by action or user, and configure the commission rate.	2 days

4.2.2 Sprint 4 Analysis

Use Case Diagram

Sprint 4 actors form the internal hierarchy *Intern* $\rightarrow$ *Employee* $\rightarrow$ *Admin* $\rightarrow$ *Super Admin* (generalization). The use cases cover task management, project management, chat, calendar, analytics, moderation, user management, role requests, categories, vouchers, audit logs and platform settings.

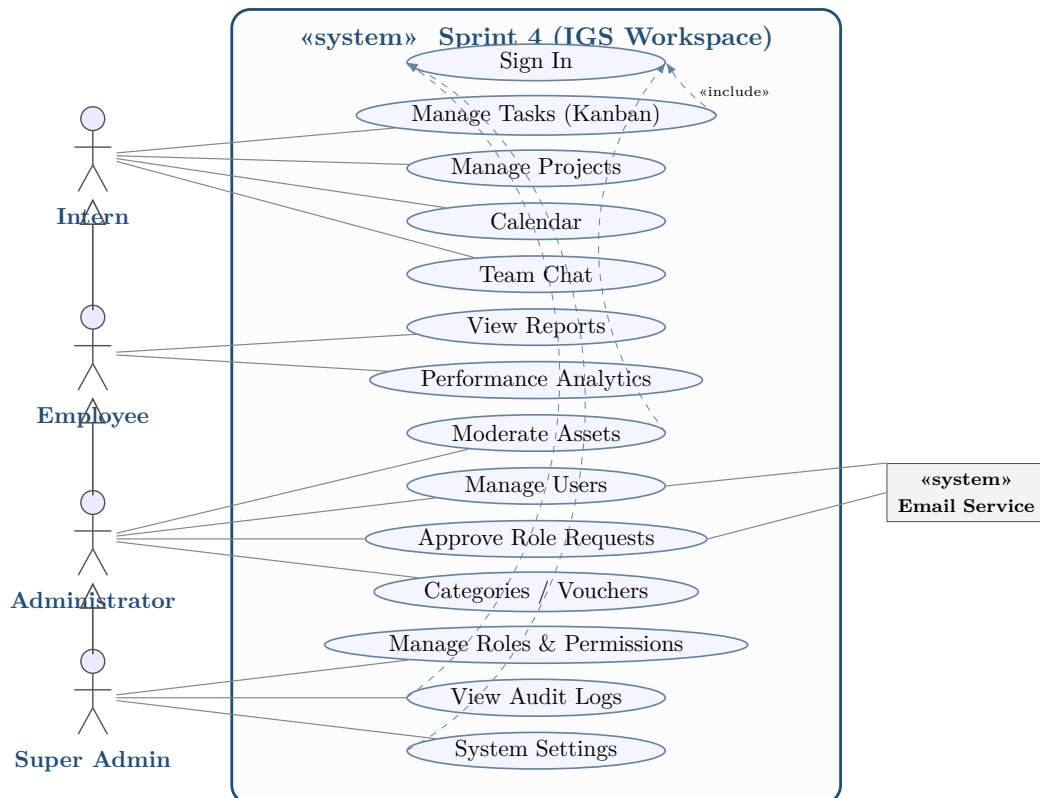


Figure 4.9: Use case diagram — Sprint 4 (Workspace)

Use Case Description — UC-05: Workspace Project and Task Management

Table 4.4: Use Case Description: Workspace Project and Task Management

UC-05 — Workspace Project and Task Management	
Use case	Workspace Project and Task Management
Actor	Employee / Admin (creation); Intern, Employee, Admin (task update)
Precondition	The actor is authenticated in IGS Workspace with at least the <i>employee</i> role.
Postcondition	The project and its tasks are persisted and progress is visible to all assigned members.
Nominal scenario	1. An employee creates a project with a name, description, deadline and color code. 2. The employee assigns team members to the project. 3. A task is created under the project with a title, priority, status and due date. 4. The task is assigned to one or more workspace members. 5. The assigned member updates the task status (<i>to-do</i> → <i>in-progress</i> → <i>done</i>). 6. The admin monitors overall progress from the project dashboard.
Alternative scenario	A1 Standalone task: A task is created without a parent project and appears on the personal task board. A2 Team chat: Members discuss the project in the dedicated Socket.IO channel in real time. E1 Insufficient role: An intern accessing an employee-only route is redirected with an access-denied banner.

Sequence Diagram — Intern Asset Request

The intern requests supervised access through the marketplace; the admin handles the request from the workspace moderation page.

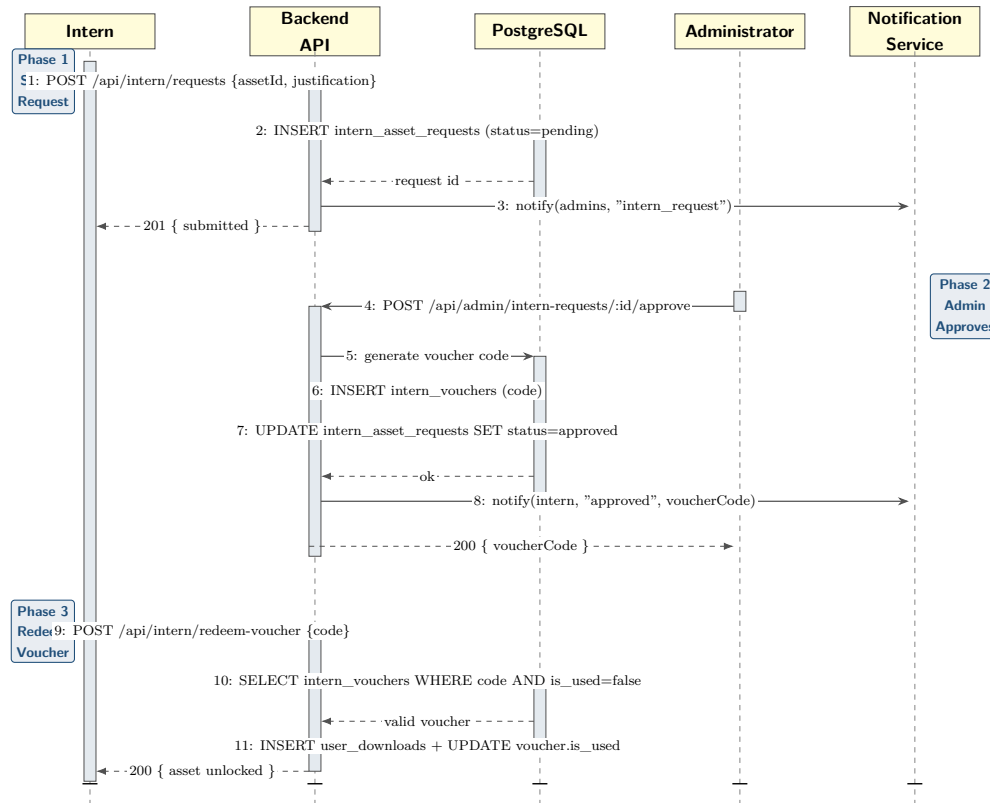


Figure 4.10: Sequence diagram — Intern asset request and supervisor approval

Class Diagram

The Sprint 4 domain covers the internal collaboration layer: **Project** and **WorkspaceTask** for project tracking, **ChatChannel** and **Message** for real-time communication, **InternAssetRequest** for supervised access and **Voucher** for discount code management.

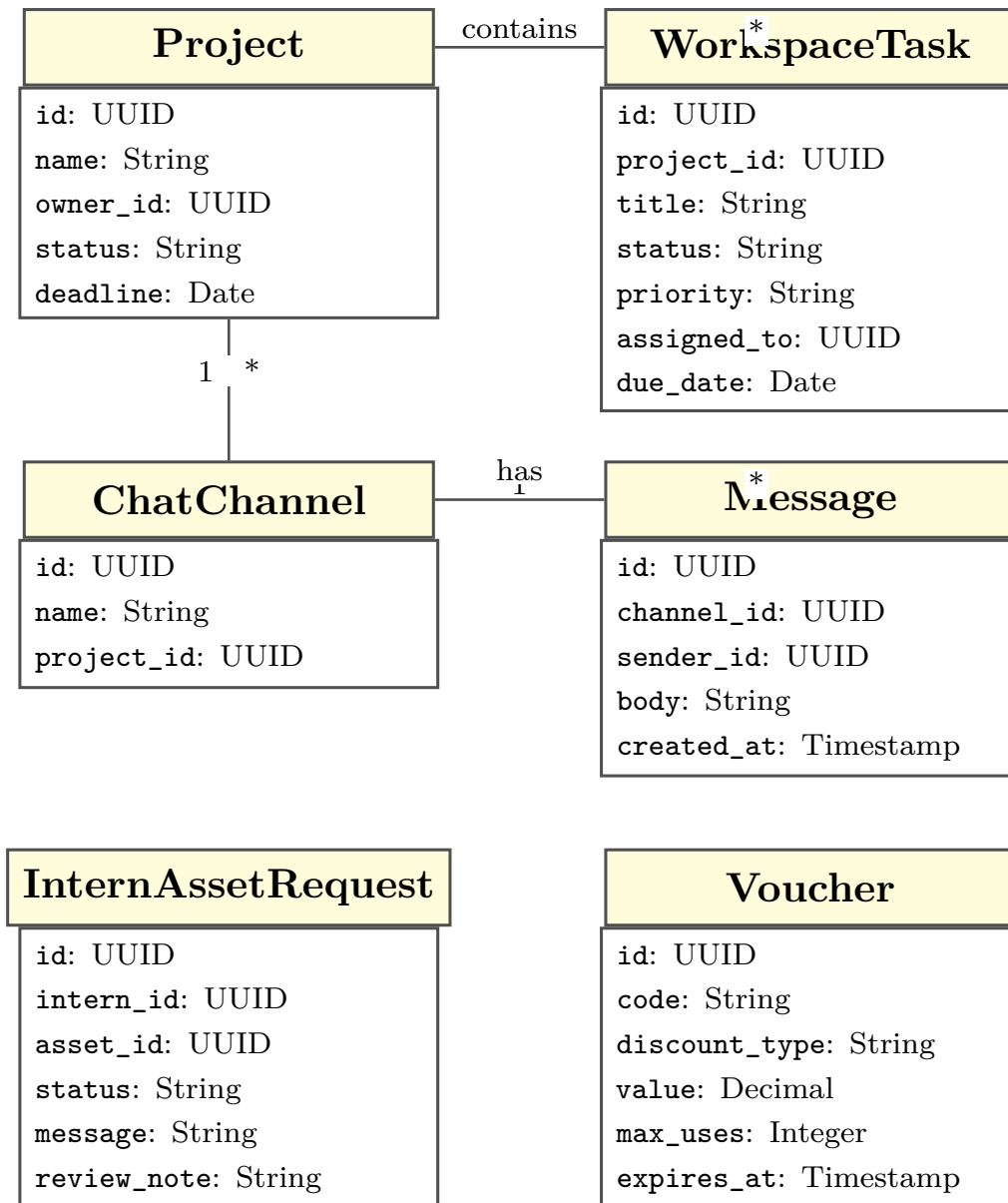


Figure 4.11: Class diagram — Sprint 4: IGS Workspace

4.2.3 Sprint 4 Realization

Application structure. The workspace is a standalone Vite/React application at `MarketPlace/IGS_Workspace/`, proxying `/api` and `/socket.io` to the backend on port 3001. The provider stack is `ThemeProvider` → `AuthProvider` → `ChatProvider`. The same `localStorage['igs_token']` written by the marketplace login is reused, so internal users switch between the two apps without re-authenticating.

Role-gated navigation. The sidebar is divided into four collapsible sections, each gated by minimum role:

Table 4.5: Workspace sidebar sections and access requirements

Section	Min Role	Pages
General	intern	Dashboard, Tasks, Team, Calendar, Chat, Projects
Analytics	employee	Reports, Performance
Management	admin	Assets, Orders, Users, Moderation, Role Requests, Categories, Vouchers
System	super_admin	Roles & Permissions, Audit Logs, Settings

The `<RequireRole>` component wraps protected routes; on access denial it redirects to `/` with `{ accessDenied: true, required: min }` in location state, so the Dashboard shows an inline banner rather than a silent empty page.

Task board, projects and calendar. The task page implements a Kanban board backed by `workspace_tasks`; tasks move between `todo`, `in_progress`, `review` and `done` columns and carry priority badges (`low / medium / high / urgent`). The projects page groups tasks under a parent project with owner, status and date range. The calendar renders task due dates and project milestones in a monthly grid.

Real-time chat. The chat page connects to a Socket.IO [25] layer; conversations are grouped by channel (project-based) or direct messages. The JWT is passed as `auth.token` at connection time and validated on every socket event.

Moderation and administration. The moderation page presents two tabs: *Pending Assets* (external submissions) and *Intern Uploads* (`pending_supervisor`). The user, order, category and voucher pages provide full CRUD interfaces. The role-requests page lists role-upgrade applications with a filter (`pending / approved / rejected`) and an inline approve/reject control that sends a notification to the applicant.

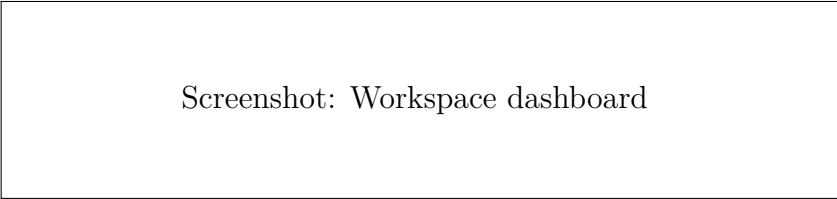
System pages. The roles-and-permissions page fetches `GET /admin/roles-with-permissions` (each role with its full permission name array and a live user count). The audit-log page exposes a filterable, paginated view of the `audit_logs` table with color-coded action badges and expandable JSON details. The settings page wires the commission rate section to `GET/PUT /api/admin/config/commission`.

Intern workflow. The intern asset request workflow is:

1. The intern opens an IGS-owned paid asset and clicks *Request Access*.

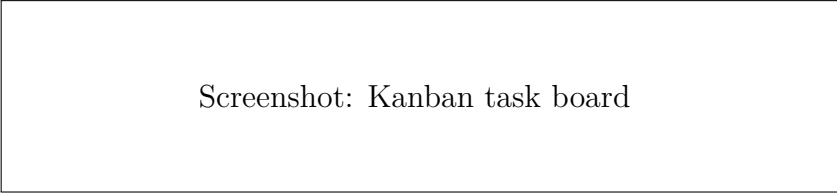
2. A modal collects a justification note (`intern_asset_requests.message`).
3. The assigned supervisor receives a notification and reviews the request from the workspace moderation page.
4. On approval, the asset is added to `user_downloads` and the intern is notified.
5. Rejected requests include a `review_note`.

Intern accounts also expire automatically: the `authenticate` middleware suspends expired interns on the next API call.



Screenshot: Workspace dashboard

Figure 4.12: IGS Workspace dashboard (placeholder)



Screenshot: Kanban task board

Figure 4.13: Kanban task board (placeholder)



Screenshot: Moderation queue

Figure 4.14: Workspace moderation page (placeholder)



Screenshot: Audit logs

Figure 4.15: Audit-log viewer (placeholder)

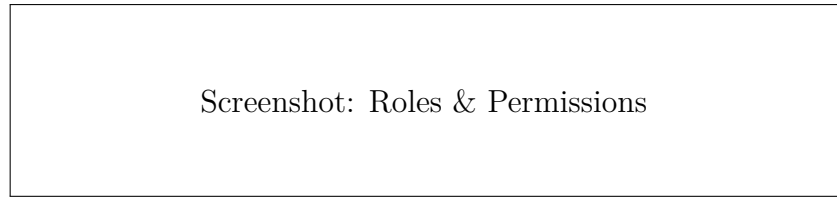


Figure 4.16: Roles and Permissions page (placeholder)

Interfaces (Sprint 4).

4.2.4 Sprint 4 Conclusion

At the end of Sprint 4 the studio team has a fully functional internal ERP. Daily operations (tasks, projects, chat, calendar) are centralised; moderators have a single queue for external and intern submissions; super-admins control the platform from a dedicated system menu and trace sensitive actions through the audit log.

Conclusion

This chapter delivered the marketplace's content production loop (Sprint 3) and the internal operations dashboard (Sprint 4). The platform is now feature-complete for both the public marketplace and the studio team. The next chapter covers Sprint 5: the AI chatbot, the security hardening layer and the proposed deployment pipeline.

Advanced Features

Sommaire

5.1	Sprint 5 Backlog	65
5.2	AI Services	66
5.2.1	AI Chatbot	66
5.2.2	Review Sentiment Analysis	67
5.2.3	Personalized Recommendations	69
5.3	Security Hardening	70
5.4	VR/AR Immersive Preview	71
5.5	Planned Deployment on Hostinger VPS	71
5.5.1	Target Infrastructure	71
5.5.2	Key Deployment Concepts	72
5.5.3	Planned CI/CD Pipeline	72

Introduction

Sprint 5 completes the platform with three AI-driven services, an immersive VR/AR preview feature and the production deployment infrastructure. The AI chatbot lets users search and explore the catalog through natural language. The sentiment analysis engine automatically classifies review text to support sellers and moderators. The content-based recommendation system surfaces personalized assets from recorded user interactions. Each AI service is presented with its model selection, training process and results, followed by its realization on the platform. The chapter closes with the WebXR immersive preview feature and the planned Hostinger deployment architecture.

5.1 Sprint 5 Backlog

Table 5.1: Sprint 5 Backlog

ID	User Story	Estimation
1	As a user, I want a chatbot widget available on every page so I can ask marketplace questions.	3 days
2	As a user, I want the chatbot to recognize my intent and reply accordingly.	2 days
3	As a user, I want my submitted review classified automatically as positive, neutral or negative.	2 days
4	As a seller, I want an aggregate sentiment summary of all reviews on my assets.	1 day
5	As a user, I want personalized asset recommendations based on my activity.	3 days
6	As a guest, I want to see trending assets when no personalization data is available.	1 day
7	As an admin, I want the API protected against brute-force with per-route rate limits.	1 day
8	As an admin, I want logout tokens to be effectively revoked, not just expired.	1 day
9	As an admin, I want uploaded images validated at the byte level to prevent MIME-confusion attacks.	1 day
10	As a maintainer, I want the platform deployed on Hostinger VPS with Nginx and PM2.	2 days

5.2 AI Services

Three AI services are integrated into the platform. Each one was selected for its fit with the available infrastructure and the scale of the data: a cloud LLM for natural language generation, a pre-trained transformer for sentiment classification, and a deterministic similarity algorithm for recommendations.

5.2.1 AI Chatbot

Analysis

Model. The chatbot relies on two models in sequence. The first is a local **TF-IDF + LinearSVC** classifier (`services/intentService.js`) that categorizes the user message into one of six intents before any network call is made. The second is **Qwen2.5-72B-Instruct**, a 72-billion-parameter instruction-tuned large language model served through the Hugging Face Inference API. It generates the final natural language response given the intent, an optional catalog search result, and a system-level prompt that confines its scope to the marketplace domain.

Training. The intent classifier was trained offline in Python using `scikit-learn`. A labeled dataset of short user utterances was prepared across six intent categories (Table 5.2). Text was vectorized with a TF-IDF transformer (up to 5 000 features, uni-grams and bigrams), and a `LinearSVC` model was fitted and serialized to `ai_training/models/inten`. The Qwen model is pre-trained by Alibaba Cloud and requires no local fine-tuning; only the system prompt was configured.

Table 5.2: Intent categories handled by the classifier

Intent	Example utterance
<code>product_search</code>	“Show me animated character models under \$10”
<code>platform_help</code>	“How do I upload my first asset?”
<code>order_status</code>	“Where is my download?”
<code>seller_info</code>	“How do commissions work?”
<code>greeting</code>	“Hello, what can you help me with?”
<code>out_of_scope</code>	“What is the weather today?”

Result. When the intent is `product_search`, the backend executes a parameterized catalog query and injects the results into the LLM prompt. For all other intents, the LLM generates a response from the system prompt alone. If the Hugging Face API

is unavailable, a rule-based fallback response is returned from a predefined template without calling any external service.

Realization

The chatbot is exposed as a floating `BotWidget` component mounted on every page. Messages are sent to `POST /api/bot/message` using `optionalAuth`, so both guests and authenticated users can interact with it. The endpoint runs the three-stage pipeline — intent classification, optional tool call, LLM generation — and streams the response back to the widget.

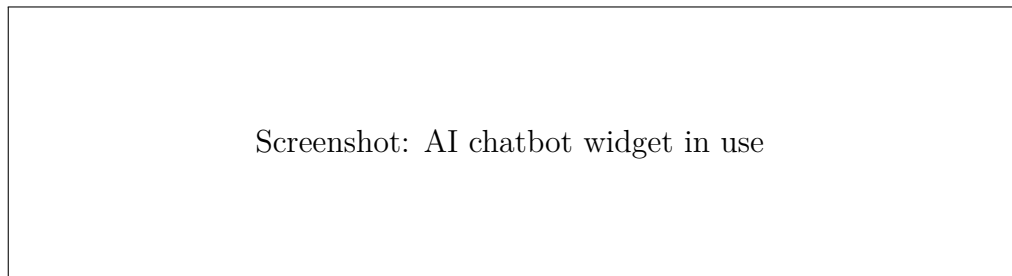


Figure 5.1: AI chatbot widget — intent-aware assistant available on every page

Conclusion

The chatbot reduces the friction of asset discovery by letting users express queries in plain language rather than constructing catalog filters manually. The local intent classifier keeps latency low and avoids sending every message to the LLM when a simple rule-based response is sufficient.

5.2.2 Review Sentiment Analysis

Analysis

Model. The primary model is **DistilBERT** fine-tuned on the Stanford Sentiment Treebank (`distilbert-base-uncased-finetuned-sst-2-english`), called via the Hugging Face Inference API. DistilBERT is a compressed version of BERT that retains 97 % of its performance at 40 % fewer parameters. It returns a label (**POSITIVE** or **NEGATIVE**) with a confidence score. Reviews with confidence below 0.65 are mapped to **neutral** to avoid low-confidence misclassification.

A local **scikit-learn LogisticRegression** model serves as fallback when the Hugging Face API is unavailable. It was trained on the 30 seed reviews from migration 043 and produces three labels: positive, neutral, and negative.

Training. The fallback model was trained offline using TF-IDF vectorization (up to 5 000 features, 1–2 n-gram range) followed by logistic regression. The small training set means the fallback is conservative; it is only activated when the primary API is unreachable, so production accuracy is driven by DistilBERT.

Result. The decision matrix governing which model is used at runtime is shown in Table 5.3.

Table 5.3: Sentiment classification — decision matrix

Condition	Engine	Output
HF API available, confidence ≥ 0.65	DistilBERT (cloud)	positive or negative
HF API available, confidence < 0.65	DistilBERT (cloud)	mapped to neutral
HF API unavailable	sklearn LogReg (local)	positive, neutral or negative

Realization

Every review submitted through `POST /api/assets/:id/reviews` is passed to `aiService.analyzeR` which returns `{sentiment, confidence}`. The result is stored in the `ai_sentiment` column of the reviews table immediately after the INSERT. On the asset detail page, each review card shows a color-coded badge (green / amber / red). The seller’s Creator Studio dashboard displays an aggregate donut chart of sentiment proportions across all reviews.

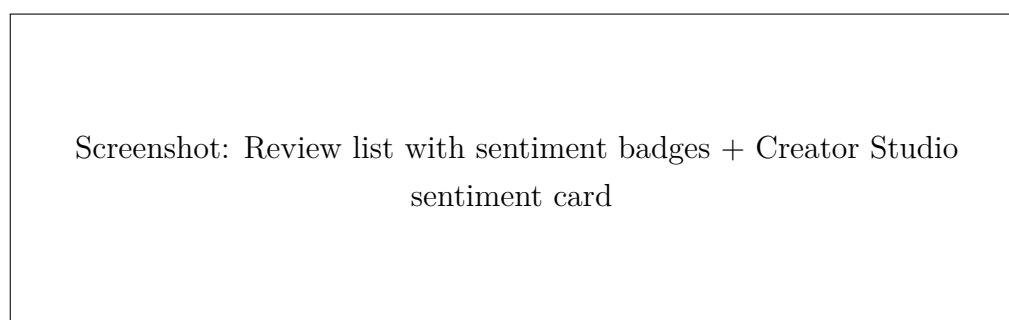


Figure 5.2: Sentiment analysis — review badges (left) and seller aggregate (right)

Conclusion

Automated sentiment classification reduces the effort required to audit large volumes of reviews. Moderators can quickly scan flagged sentiment distributions and sellers receive actionable feedback without reading every individual review.

5.2.3 Personalized Recommendations

Analysis

Model. The recommendation service uses **content-based filtering** with **Jaccard tag similarity**. No model is trained; the algorithm is fully deterministic. Each asset is represented by its tag set, and candidate assets are scored against a user profile derived from their interaction history. The score formula combines tag similarity, category affinity, keyword overlap, and a log-scaled popularity boost.

Signal collection. Four interaction tables contribute to the user profile, weighted by intent strength:

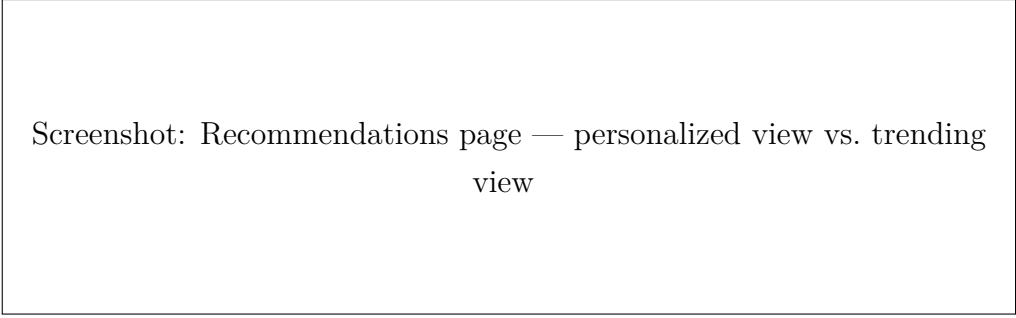
Table 5.4: User interaction signals and weights

Source	Weight	Rationale
user_downloads	3	Direct asset acquisition
asset_likes	5	Strongest explicit preference signal
asset_ratings	1–5	Weighted by star value
order_items (paid)	4	Paid purchase is high-confidence signal

Result. Authenticated users with at least two signals receive a personalized list ranked by Jaccard score. A diversity pass demotes near-duplicate assets (tag overlap > 70%) to keep the list varied. Users with no signals (guests and new accounts) receive the trending list from `getTrending()`, and the frontend displays “Trending Now” instead of “Recommended for You”.

Realization

The endpoint `GET /api/recommendations` is wrapped in `optionalAuth` so guests receive trending results without a login wall. The response includes a `coldStart` boolean that the `Hot.jsx` page reads to switch the section heading. On the frontend, the recommendation row is visually identical to a standard catalog row, so users see relevant content immediately without any onboarding step.



Screenshot: Recommendations page — personalized view vs. trending view

Figure 5.3: Recommendations page — personalized (warm user) and trending (cold start)

Conclusion

Content-based filtering with Jaccard similarity provides meaningful recommendations without requiring a large user base or a separate training pipeline. The cold-start fallback ensures guests also receive value from the first visit, and the signal weighting can be adjusted without retraining any model.

5.3 Security Hardening

Three independent middlewares protect the API against the most common attack vectors:

- **Rate limiting** (`middleware/rateLimiters.js`): login is limited to 10 attempts per 15-minute window per IP, registration to 5 accounts per hour, and asset uploads have a separate lower ceiling. Exceeding any limit returns `429 Too Many Requests`.
- **Token blacklist** (`services/tokenBlacklist.js`): on logout, the JWT is added to an in-memory set checked by the `authenticate` middleware before signature verification. This enforces immediate revocation without requiring token expiry.
- **Magic-byte validation**: thumbnail uploads are inspected at the byte level against the signatures of PNG (89 50 4E 47), JPEG (FF D8 FF), WEBP (52 49 46 46) and GIF (47 49 46 38). A renamed file with mismatched binary content is rejected with `400 Bad Request` before storage.

The OWASP Top Ten checklist [33] was used to confirm coverage: broken access control (RBAC guards), cryptographic failures (bcrypt + signed JWT + HTTPS), injection (parameterized queries throughout), security misconfiguration (CORS whitelist, Helmet headers, env-var validation at boot), and authentication failures (rate-limited login, token blacklist, automatic intern expiry).

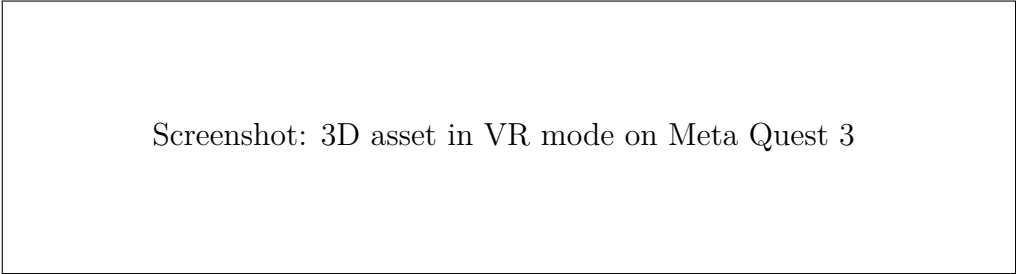
5.4 VR/AR Immersive Preview

The asset detail page exposes an immersive inspection mode for compatible three-dimensional models through the **WebXR Device API**. When the user clicks the VR or AR button, the Three.js renderer switches to stereo mode (`renderer.xr.enabled = true`) and the `VRButton` or `ARButton` component from `three/examples/jsm/webxr/` injects the browser's native XR session controls.

In VR mode, the asset is rendered at real-world scale inside the headset, allowing the user to inspect proportions, surface detail and file quality before purchasing. In AR mode, the asset is overlaid on the physical environment captured by the passthrough cameras, which is useful for architecture and environment assets.

The platform was tested on a **Meta Quest 3** headset connected to the same local network as the development server. The test workflow: start the dev server with HTTPS (`npm run dev`), open `https://<local-IP>:5173` in the Meta Browser, accept the self-signed certificate, and open any 3D model detail page to trigger the VR/AR button.

Two post-processing passes (**Bloom** and **Bokeh** via **EffectComposer**) are disabled when `renderer.xr.isPresenting` is true, as they cause rendering errors in stereo mode. `OrbitControls.update()` is also skipped during XR sessions for the same reason.



Screenshot: 3D asset in VR mode on Meta Quest 3

Figure 5.4: WebXR immersive preview — 3D model inspected in VR on Meta Quest 3

5.5 Planned Deployment on Hostinger VPS

The platform is intended for production deployment on a **Hostinger VPS**. The deployment configuration files (`nginx.conf`, `ecosystem.config.js`, GitHub Actions workflow) are already prepared in the repository pending the final release after the defense.

5.5.1 Target Infrastructure

Three independent tiers handle different concerns:

- **Hostinger VPS**: hosts the Node.js backend (Express API) and serves the two built React applications (Marketplace and Workspace) as static files.
- **Supabase** (cloud-hosted): handles the PostgreSQL database and the asset file buckets. No database instance is needed on the VPS.
- **Hugging Face Inference API**: serves the LLM and DistilBERT models remotely; no GPU hardware is required on the server.

5.5.2 Key Deployment Concepts

- **Virtual Private Server (VPS)**: an isolated virtual machine running Ubuntu 22.04 LTS, providing full root access without the cost of dedicated hardware.
- **Nginx (reverse proxy)**: terminates HTTPS, serves the static React builds from `/var/www/`, and forwards `/api/*` and `/socket.io` traffic to the Express backend on port 3001.
- **PM2 (process manager)**: keeps the Node.js process alive, restarts it on crash, enables zero-downtime reloads on new deploys, and manages log rotation.
- **Let's Encrypt + Certbot**: free SSL certificates renewed automatically every 90 days; HTTPS is mandatory for WebXR features to work in the browser.
- **Environment variables (.env)**: keep secrets (JWT secret, database credentials, Stripe keys, Hugging Face token) out of the source code and out of the Git history entirely.

5.5.3 Planned CI/CD Pipeline

A GitHub Actions workflow (`.github/workflows/cd.yml`) automates the release on every push to `main`:

1. Install dependencies and run lint and integration tests.
2. Build both React frontends with `npm run build`.
3. SSH into the Hostinger VPS, pull the latest code, copy the built `dist/` folders to `/var/www/`, and call `pm2 reload igs-backend` for a zero-downtime restart.

Deployment secrets (`VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY`) are stored in GitHub repository secrets and never appear in workflow logs.

Conclusion

This chapter presented the three AI services that enrich the platform: a three-stage chatbot pipeline using intent classification and LLM generation, an automatic review sentiment engine based on DistilBERT with a local fallback, and a content-based recommendation system using Jaccard tag similarity. It also covered the WebXR immersive preview feature validated on Meta Quest 3, the security hardening layer, and the planned Hostinger deployment architecture. Together these layers complete the platform and make it production-ready for the final release.

General Conclusion

IGS Marketplace was successfully designed and developed over eight sprints, delivering Tunisia's first dedicated 3D digital asset platform. The system provides a six-level Role-Based Access Control system, an interactive in-browser 3D asset viewer, Stripe payment integration, AI-powered review sentiment analysis, a personalized recommendation engine, a floating chatbot assistant powered by Qwen2.5-72B-Instruct, and a companion internal workspace application — all within a cohesive full-stack ecosystem built on React, Node.js/Express, and PostgreSQL.

Several key technical insights emerged throughout the project. Designing the permission system during Sprint 0 and enforcing it consistently across all subsequent sprints was the single most impactful architectural decision — retrofitting access control would have been significantly more costly. The three-tier layered architecture (Routes → Controllers → Repositories) proved highly maintainable for a REST API at this level of complexity, and applying Scrum as a solo developer, with supervisors occupying the Product Owner and Scrum Master roles, effectively structured six months of work into reviewable, deliverable-focused increments.

Looking ahead, several directions are planned for the platform's continued evolution: a React Native mobile client, a full automated test suite using Jest and Supertest, expanded AI capabilities including streaming chatbot responses and AI-based tag suggestions, a digital asset licensing framework where sellers can define usage rights, and production deployment on a cloud provider with the Docker-based CI/CD pipeline fully active.

Webography

- [1] Unity Technologies. *Unity Asset Store*.
<https://assetstore.unity.com/> [Accessed April 2026]
- [2] Epic Games. *Fab — The unified content marketplace by Epic Games*.
<https://www.fab.com/> [Accessed April 2026]
- [3] Sketchfab (Epic Games). *Sketchfab — Publish & Find 3D Models Online*.
<https://sketchfab.com/> [Accessed April 2026]
- [4] CGTrader. *CGTrader — 3D Model Marketplace and Computer Graphics Forum*.
<https://www.cgtrader.com/> [Accessed April 2026]
- [5] Threedy.ai. *Cosmos — AI-powered 3D content platform*.
<https://www.threedy.ai/> [Accessed April 2026]
- [6] Adobe Inc. *Mixamo — Free animated 3D characters*.
<https://www.mixamo.com/> [Accessed April 2026]
- [7] Microsoft. *Visual Studio Code — Code Editing Redefined*.
<https://code.visualstudio.com/> [Accessed May 2026]
- [8] GitHub, Inc. *GitHub — Where the world builds software*.
<https://github.com/> [Accessed May 2026]
- [9] GitHub, Inc. *GitHub Actions Documentation*.
<https://docs.github.com/en/actions> [Accessed May 2026]
- [10] Docker, Inc. *Docker — Accelerated Container Application Development*.
<https://www.docker.com/> [Accessed May 2026]
- [11] Hostinger International. *Hostinger — Web Hosting and VPS*.
<https://www.hostinger.com/> [Accessed May 2026]
- [12] JGraph Ltd. *draw.io — Free Online Diagram Software*.
<https://www.drawio.com/> [Accessed May 2026]
- [13] OpenJS Foundation. *ESLint — Find and fix problems in JavaScript code*.
<https://eslint.org/> [Accessed May 2026]

- [14] Tailwind Labs. *Tailwind CSS — A utility-first CSS framework*.
<https://tailwindcss.com/> [Accessed May 2026]
- [15] Meta Platforms, Inc. *Meta Quest 3 — Mixed Reality Headset*.
<https://www.meta.com/quest/quest-3/> [Accessed May 2026]
- [16] Tool logos used throughout this report were obtained from the official websites of the respective projects: React, Node.js, Express, Vite, PostgreSQL, Supabase, Stripe, Socket.IO, Three.js, Hugging Face, Docker, GitHub, GitHub Actions, draw.io, ESLint, Tailwind CSS, JWT, Hostinger, Meta Quest and L^AT_EX.
<https://react.dev/> ; <https://nodejs.org/> ; <https://threejs.org/> [Accessed May 2026]
- [17] Meta Platforms. *React — The library for web and native user interfaces*.
<https://react.dev/> [Accessed April 2026]
- [18] Vite contributors. *Vite — Next generation frontend tooling*.
<https://vitejs.dev/guide/> [Accessed April 2026]
- [19] OpenJS Foundation. *Node.js Official Documentation*.
<https://nodejs.org/en/docs/> [Accessed April 2026]
- [20] OpenJS Foundation. *Express.js Application Guide*.
<https://expressjs.com/en/guide/routing.html> [Accessed April 2026]
- [21] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*.
<https://www.postgresql.org/docs/16/> [Accessed April 2026]
- [22] Supabase Inc. *Supabase Documentation — Storage*.
<https://supabase.com/docs/guides/storage> [Accessed April 2026]
- [23] Supabase Inc. *Supabase Documentation — Auth*.
<https://supabase.com/docs/guides/auth> [Accessed April 2026]
- [24] Stripe, Inc. *Stripe API Reference — Payment Intents*.
https://stripe.com/docs/api/payment_intents [Accessed April 2026]
- [25] Socket.IO contributors. *Socket.IO v4 Documentation*.
<https://socket.io/docs/v4/> [Accessed April 2026]
- [26] Google LLC. *Model Viewer — Web Component for 3D Models*.
<https://modelviewer.dev/> [Accessed April 2026]
- [27] Three.js contributors. *Three.js — JavaScript 3D Library*.
<https://threejs.org/docs/> [Accessed April 2026]

- [28] Hugging Face. *Inference API Documentation*.
<https://huggingface.co/docs/api-inference> [Accessed April 2026]
- [29] Schwaber, K. & Sutherland, J. *The Scrum Guide*. November 2020.
<https://scrumguides.org/scrum-guide.html> [Accessed April 2026]
- [30] Object Management Group. *UML 2.5.1 Specification*. 2017.
<https://www.omg.org/spec/UML/2.5.1> [Accessed April 2026]
- [31] Jones, M. et al. *RFC 7519 — JSON Web Token (JWT)*. IETF, 2015.
<https://datatracker.ietf.org/doc/html/rfc7519> [Accessed April 2026]
- [32] Ferraiolo, D. & Kuhn, R. *Role-Based Access Control*. NIST, 2009.
- [33] OWASP Foundation. *OWASP Top Ten — Web Application Security Risks*.
<https://owasp.org/www-project-top-ten/> [Accessed April 2026]
- [34] P Market Research. *Worldwide 3D Marketplace Models Market Research 2024 — by Type, Application, Participants and Countries, Forecast to 2030*. 2024.
<https://pmarketresearch.com/worldwide-3d-marketplace-models-market-research-2024>
[Accessed April 2026]
- [35] Citius Research. *3D Project Market Report*. 2024.
<https://citiusresearch.com/publication-details/3d-project-market-report> [Accessed April 2026]
- [36] Roques, P. *UML 2 par la Pratique — Études de cas et exercices corrigés*. 8^e édition. Eyrolles, 2014.

Abstract

This end-of-studies project presents the design and development of **IGS Marketplace**, a full-stack digital asset platform built for **Inherited Games Studio**, a Tunisian video-game company. The platform centralises the publication, purchase and download of game-ready 3D assets (models, textures, HDRIs, shaders, scripts, audio, VFX, animations) and provides secure authentication, role-based access control, an interactive 3D viewer, Stripe checkout, a seller creator studio, and AI-powered features (chatbot, review sentiment analysis, content recommendations). It also integrates **IGS Workspace**, an internal ERP dashboard for collaboration, task management and moderation. The solution is built with **React**, **Node.js / Express** and **PostgreSQL (Supabase)**, following the **Scrum** methodology.

Keywords: Digital Asset Marketplace, 3D, React, Node.js, PostgreSQL, RBAC, Scrum, AI.

Résumé

Ce projet de fin d'études présente la conception et le développement de **IGS Marketplace**, une plateforme web full-stack réalisée pour **Inherited Games Studio**, société tunisienne de développement de jeux vidéo. La plateforme centralise la publication, l'achat et le téléchargement d'assets 3D (modèles, textures, HDRIs, shaders, scripts, audio, VFX, animations) et offre authentification sécurisée, contrôle d'accès par rôles, visualiseur 3D interactif, paiement Stripe, espace créateur et services d'IA (chatbot, analyse de sentiment, recommandations). Elle intègre également **IGS Workspace**, un tableau de bord ERP interne. La solution repose sur **React**, **Node.js / Express** et **PostgreSQL (Supabase)**, suivant la méthodologie **Scrum**.

Mots-clés : Marketplace d'assets numériques, 3D, React, Node.js, PostgreSQL, RBAC, Scrum, IA.

ملخص

يقدم هذا المشروع تصميم وتطوير منصة **IGS Marketplace** لفائدة شركة **Studio Games Inherited** التونسية المتخصصة في تطوير ألعاب الفيديو. تتيح المنصة نشر وبيع وتنزيل الأصول الرقمية ثلاثية الأبعاد، مع نظام مصادقة آمن، صلاحيات قائمة على الأدوار، عارض ثلاثي الأبعاد، دفع عبر **Stripe** فضاء للبائع، وخدمات ذكاء اصطناعي. ويتضمن المشروع أيضا **IGS Workspace**، لوحة تحكم داخلية لإدارة الفريق والمهام. تعتمد الحل على **React**، **Node.js / Express** و **PostgreSQL (Supabase)**، وفق منهجية **Scrum**.

الكلمات المفتاحية: منصة أصول رقمية، أبعاد ثلاثية، **React**، **Node.js**، **PostgreSQL**، **RBAC**، **Scrum**، ذكاء اصطناعي.
